

AQST| Monday 23/03/2026, US-VNUHCM

AI for quantum & Quantum for AI & Quantum AI

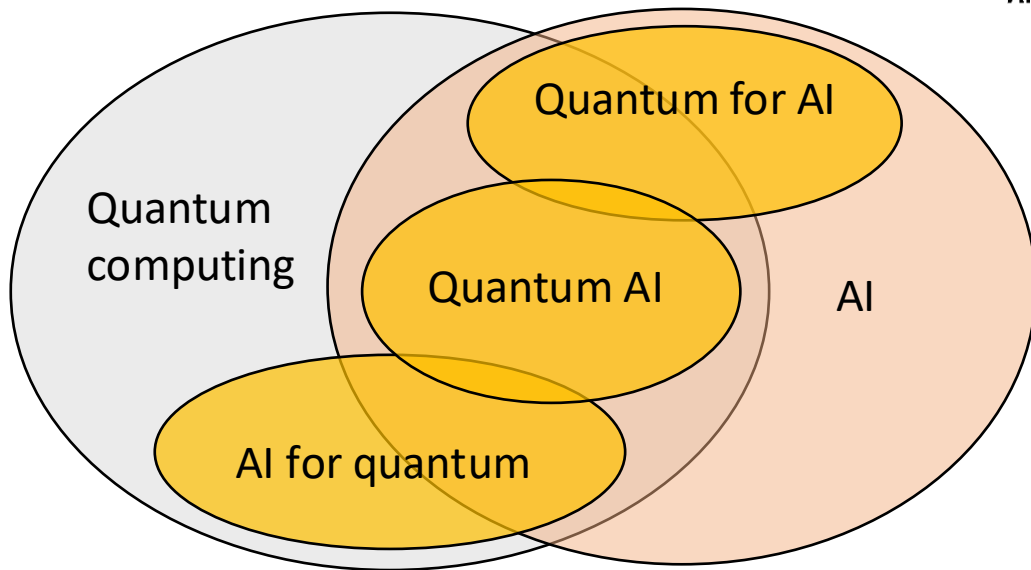
PhD. Vu Tuan Hai

University of Information Technology (UIT), VNUHCM

Email: haivt@uit.edu.vn

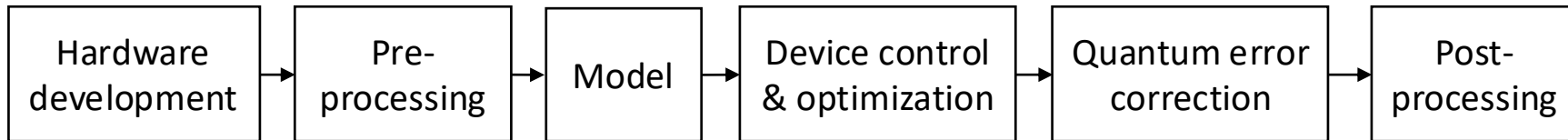
Outline

1. AI for quantum
2. Quantum for AI
3. Quantum AI

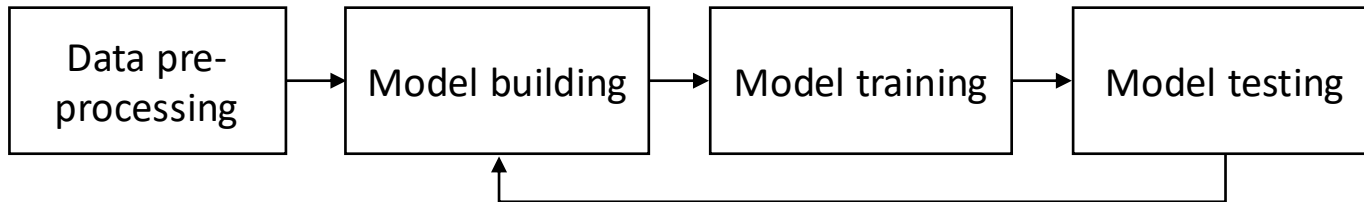


Overview

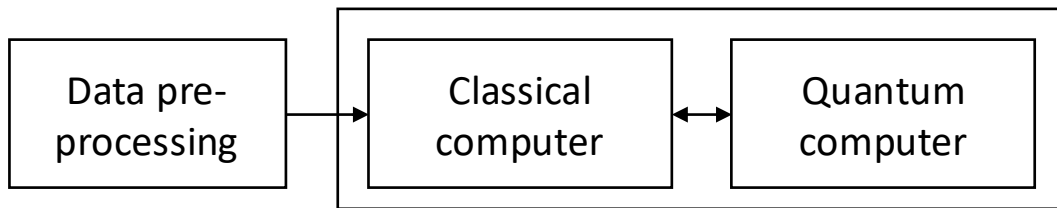
Quantum



AI

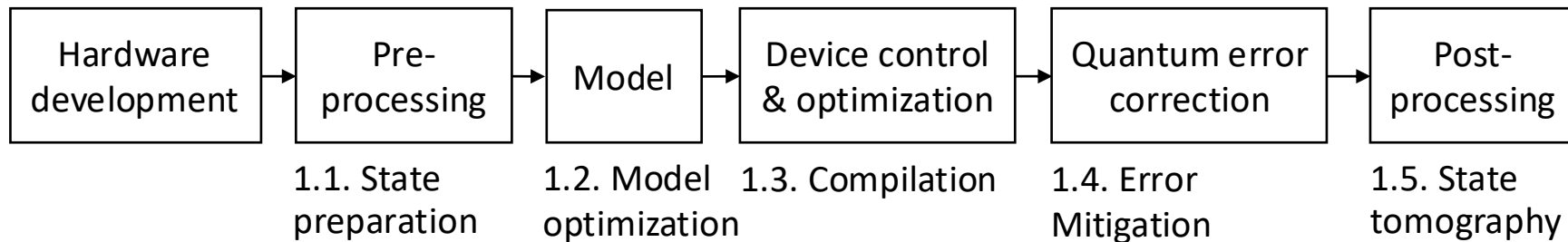


Quantum AI

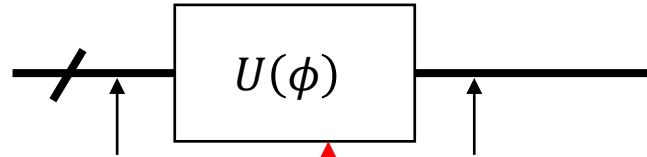


1. AI for quantum

Quantum



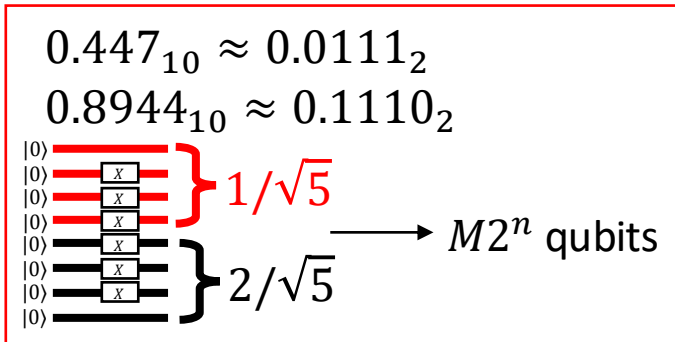
1.1. State preparation



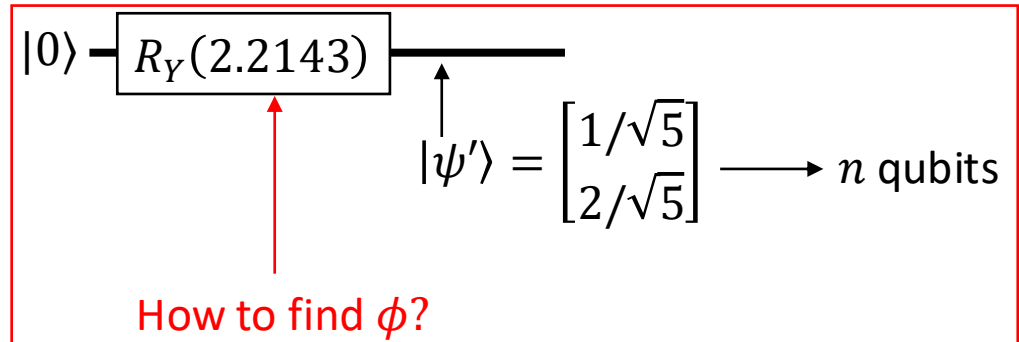
$|\psi_0\rangle = |0\rangle^{\otimes n}$ $|\psi'\rangle = U(\phi)|\psi_0\rangle$ with $|\psi'\rangle$ is the prepared/embedded state

How to prepare $U(\phi)$?

For example: $|\psi'\rangle = \begin{bmatrix} 1/\sqrt{5} & 2/\sqrt{5} \end{bmatrix}$



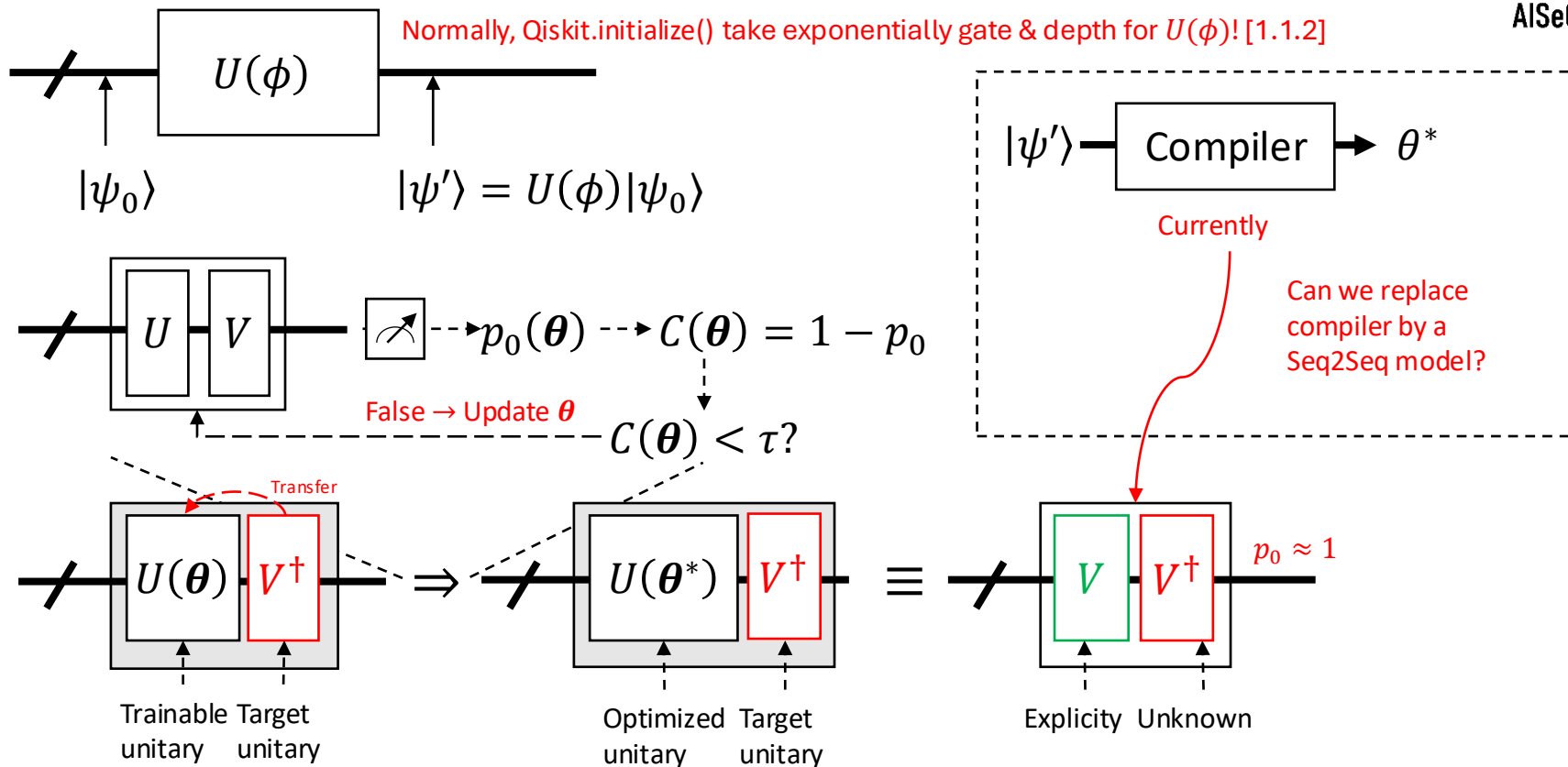
Basic encoding



How to find ϕ ?

Amplitude encoding

1.1. State preparation



[1.1.2] Iten et al., Quantum circuits for isometries (2016). [Phys. Rev. A 93, 032318](https://arxiv.org/abs/1601.06947)

[1.1.3] Hai, V. T., Viet, N. T., & Ho, L. B. (2024). *SoftwareX*, 26, 101726. <https://doi.org/10.1016/j.softx.2024.101726>

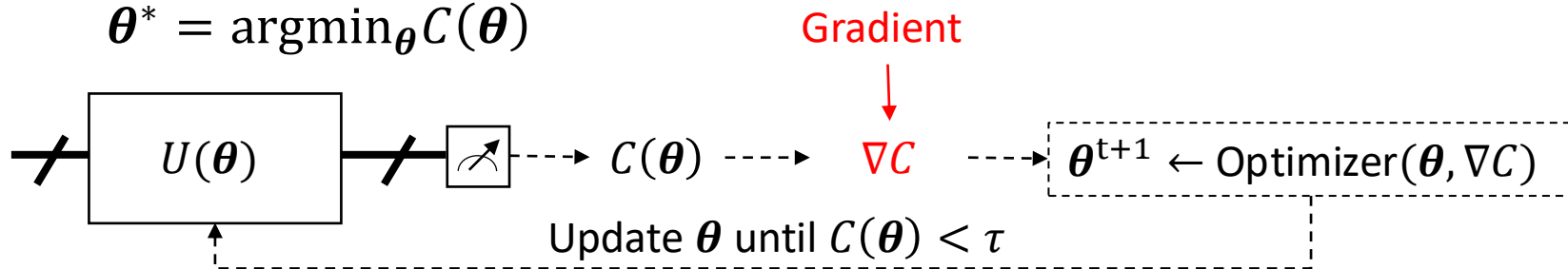
1.2. Model optimization

Considering a PQC $U(\boldsymbol{\theta})$ and cost function:

$$C(\boldsymbol{\theta}) = \langle \mathbf{0} | U^\dagger(\boldsymbol{\theta}) \hat{B} U(\boldsymbol{\theta}) | \mathbf{0} \rangle$$

We want to find:

$$\boldsymbol{\theta}^* = \operatorname{argmin}_{\boldsymbol{\theta}} C(\boldsymbol{\theta})$$



1.2.1. Gradient

Computing PQC's gradient is the key in quantum optimization:

$$\nabla C = [\partial_{\theta_0} C \quad \dots \quad \partial_{\theta_{m-1}} C]^\top$$

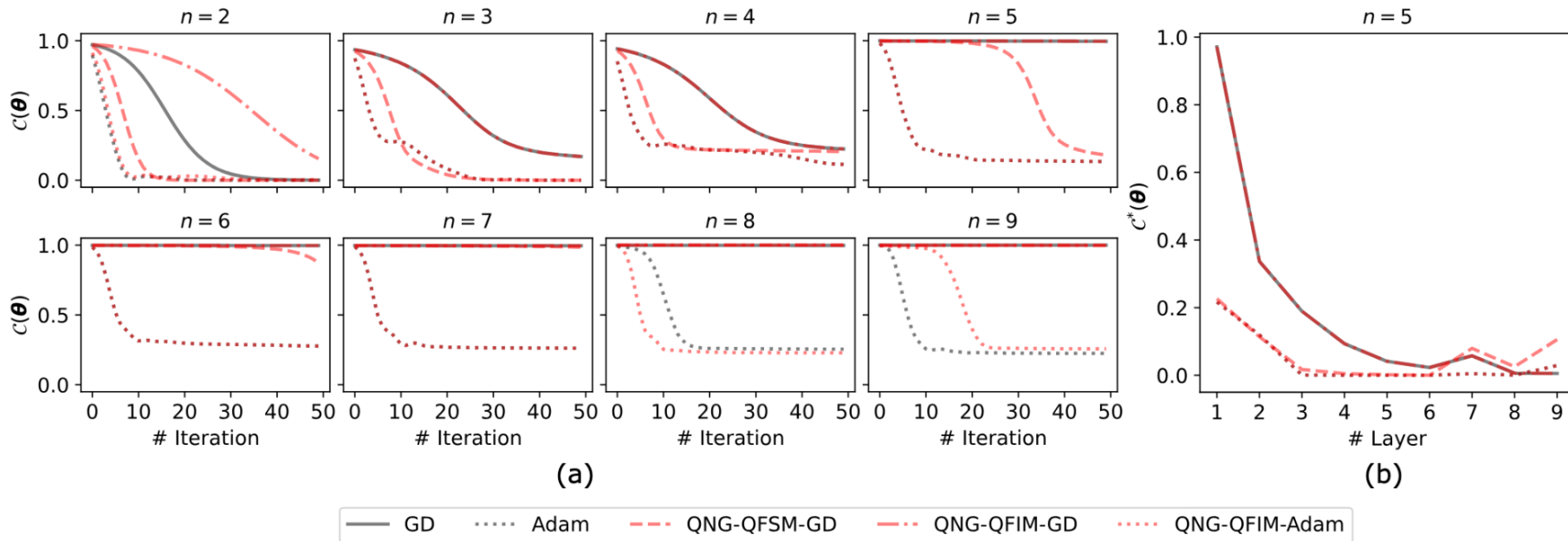
The partial derivative can be computed through parameter-shift rule **[1.17]**:

$$\frac{\partial C}{\partial \theta_j} = \sum_{k=1}^R d_k \left(C(\theta_j^{+\epsilon_k}) - C(\theta_j^{-\epsilon_k}) \right)$$

and $C(\theta_j^{+\epsilon_k}) \equiv C(\boldsymbol{\theta} + \mathbf{e}_j \epsilon_k)$, $\mathbf{e}_j$ is the j^{th} unit vector, ϵ_k is the shift-value

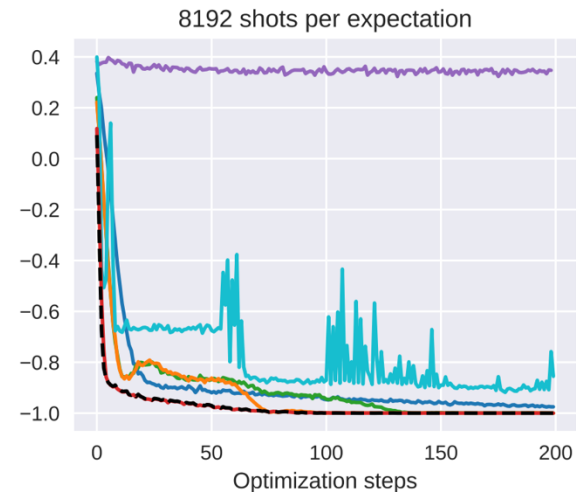
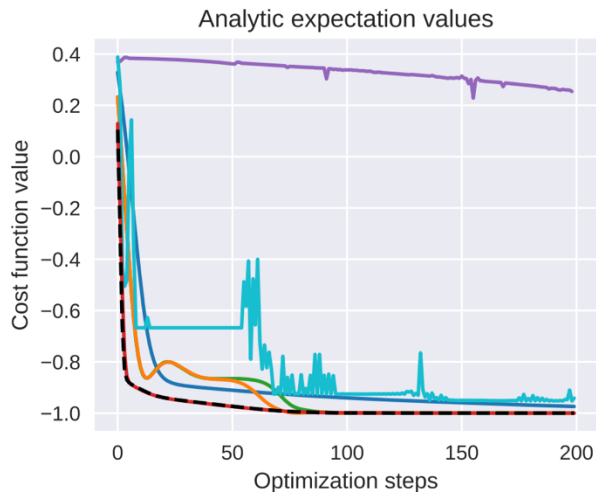
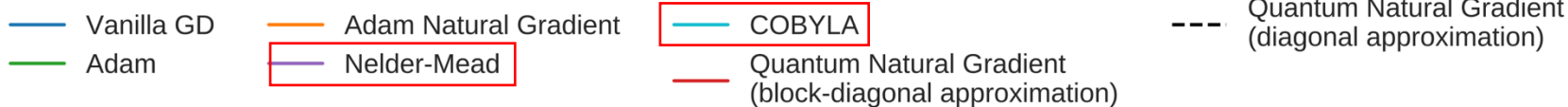
1.2.2. Optimizer

Quantum optimization is still heavily depend on classical optimizers such as Adam



1.2.2. Optimizer

Note that we can also use gradient-free optimizers: Nelder-Mead, COBYLA [1.2.3]



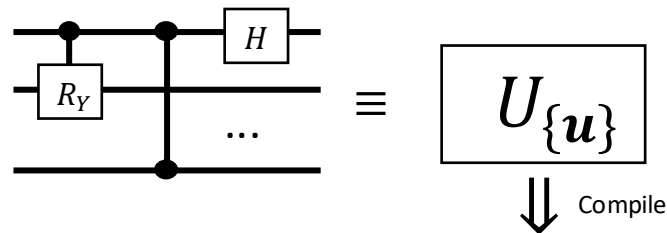
1.3. Compilation

There is a large gap between algorithms and set of instructions we can perform on real q

Problem 1 (a): The native gate set is diff

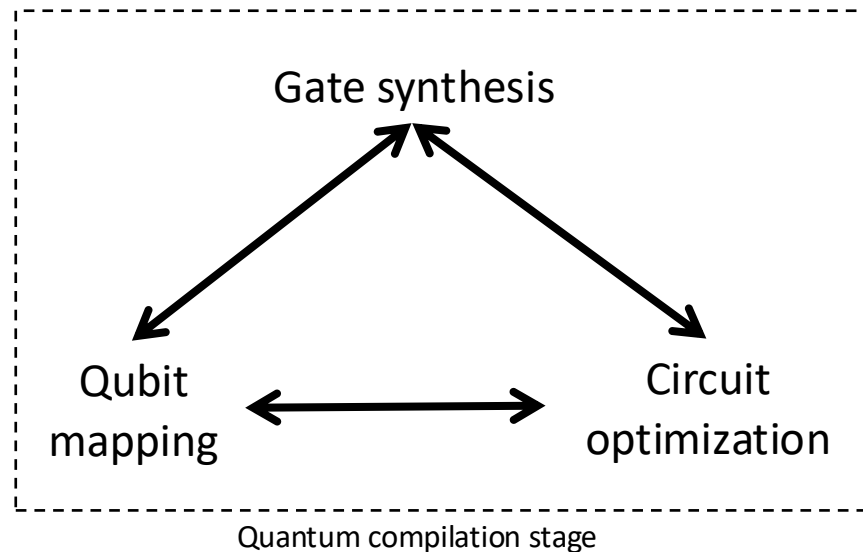
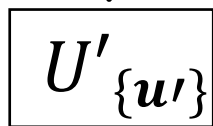
No CX, R_X or CR_Y	$\{CZ, X, R_Z, SX, ID\}$	IBM series
No CX, R_Y or CR_Y	$\{R_X, R_Z, ISWAP\}$	Rigetti Ankaa-3

If we have this example circuit:



$$u = \{R_{X,Y,Z}, CX\}$$

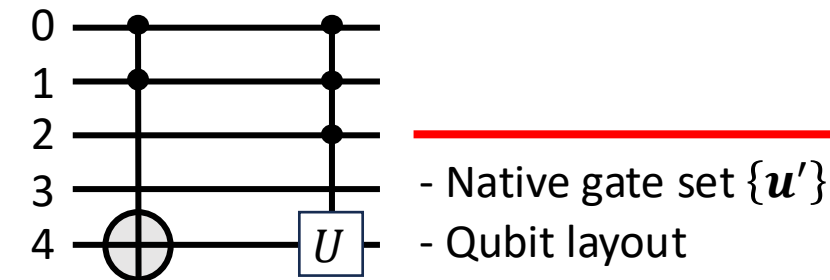
$$u' = \{R_{X,Z}, ISWAP\}$$



1.3. Compilation

Gap between algorithms and set of instructions we can perform on real qc

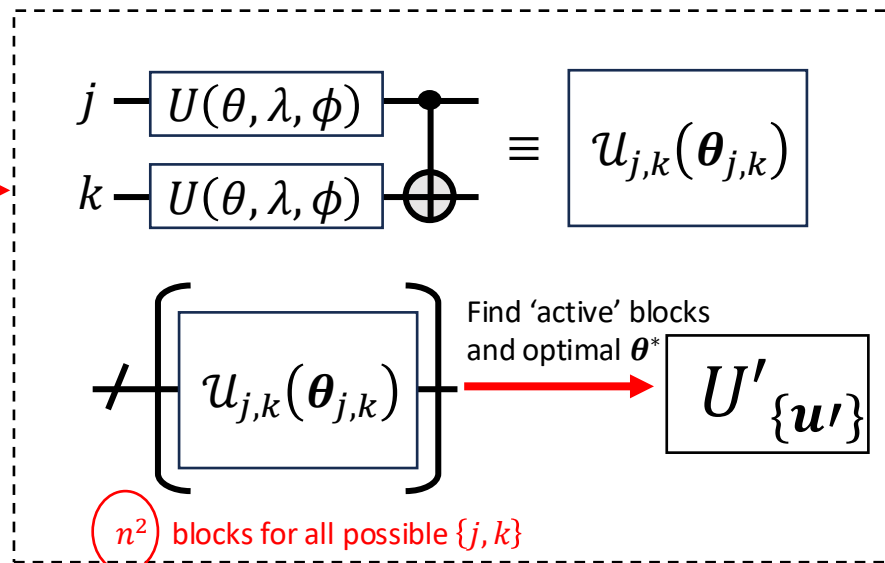
Problem 1 (b): Some gates are not exist



Toffoli gate Multi-control gate

Number of qubits	2	3	4	5	n_{CX}
Iterative unentangling [21]	8	62	344	1642	$2 \cdot 4^n - (2n + 3) \cdot 2^n + 2n$
Givens rotations [20, 27]	4	64	536	4156	$\approx 8 \cdot 4 \cdot 4^n$
Recursive CSD [22]	14	92	504	2544	$\frac{1}{2} \cdot n \cdot 4^n - \frac{1}{2} \cdot 2^n$
Recursive CSD (optimized) [23]	4	26	118	494	$\frac{1}{2} \cdot 4^n - \frac{1}{2} \cdot 2^n - 2$
QSD [21, 27]	6	36	168	720	$\frac{3}{4} \cdot 4^n - \frac{3}{2} \cdot 2^n$
QSD (optimized)[21]	3	20	100	444	$(23/48) \cdot (4^n) - \frac{3}{2} \cdot 2^n + \frac{4}{3}$
Sequential optimization ^a	3	15	63	267	Not available
Theoretical Lower Bounds[18]	3	14	61	252	$\frac{1}{4} (4^n - 3n - 1)$

Note that there is a theoretical limit (lower bound)



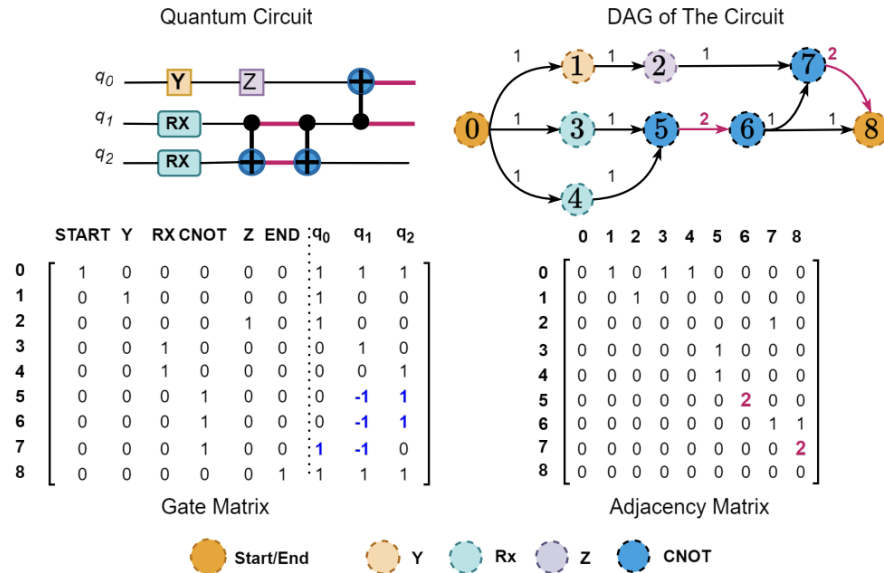
SQUANDER package [1.13]

Reduce this complexity
by AI models?

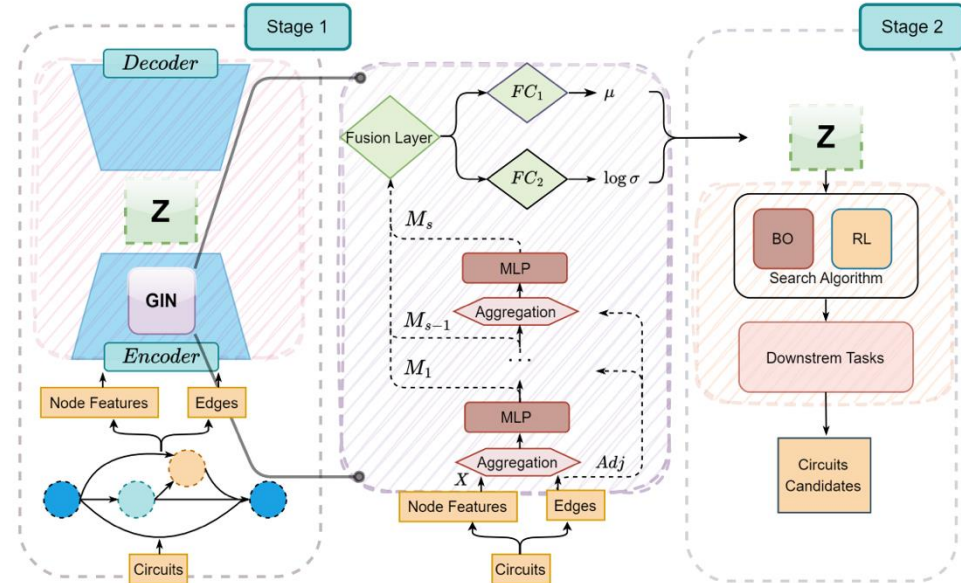
1.3. Compilation

Can we speed-up compilation process by AI models?

- Quantum architecture search (finding the best ansatz) [1.3.3]



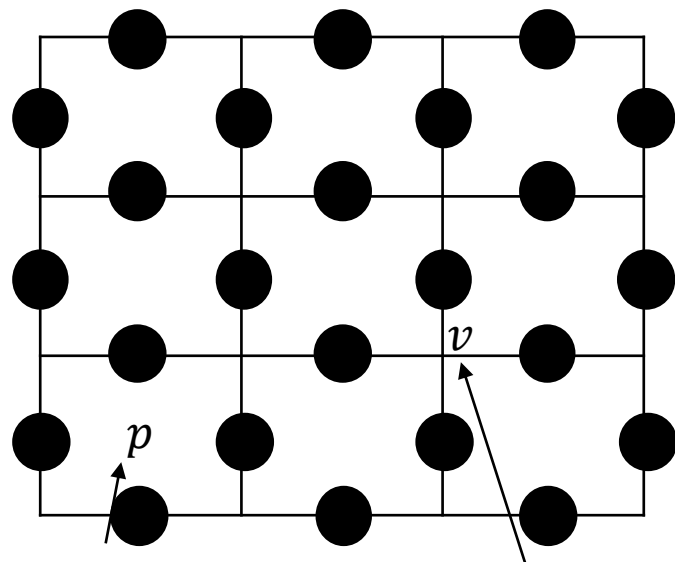
(a) Architecture encoding scheme



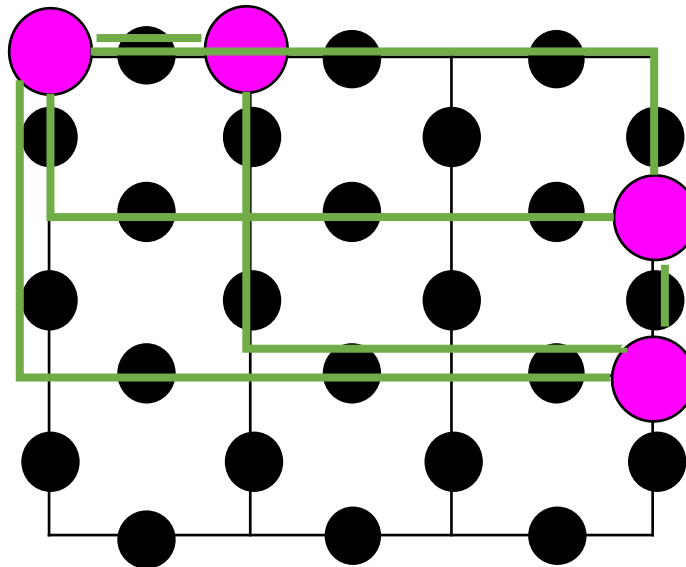
(b) Representation learning and search process

1.4. Quantum error correction

MWPM decoder (Minimum Weight Perfect Matching) used in Surface code syndrome $\rightarrow$ error qubits



Stabilizer operator Z is surrounded by qubitS

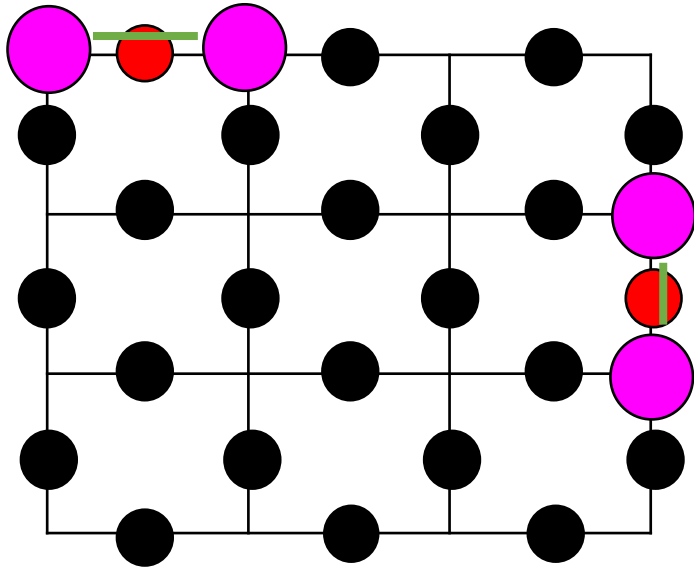
Stabilizer operator X 

Detected
syndrome!

1.4. Quantum error correction

MWPM decoder (Minimum Weight Perfect Matching)

Find shortest pairs through Blossom algorithm



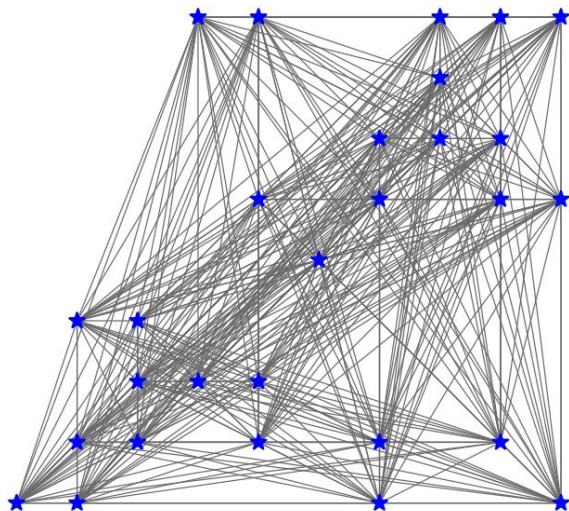
Matched



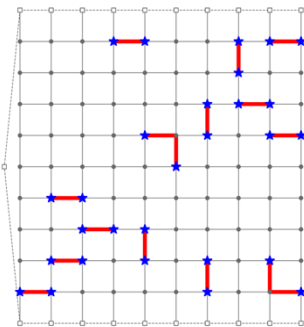
1.4. Quantum error correction

Complexity: $O(N^3 \log N)$ with N is the number of syndrome.

How to reduce it?



CNN: look graph, then guess the ver/horiz line
 Graph NN: perfect candidate?
 Transformer: ???

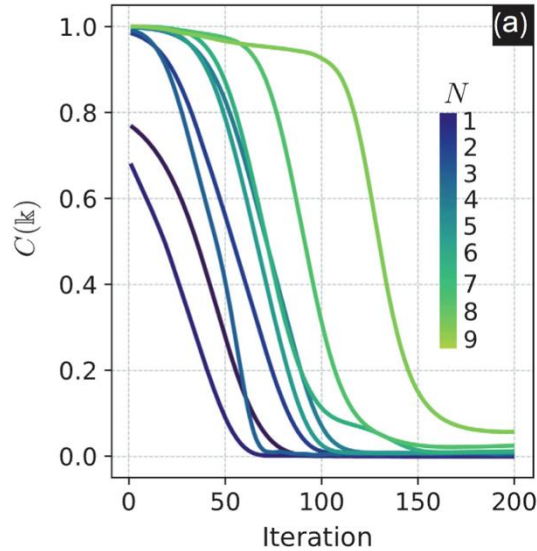


1.5. Tomography

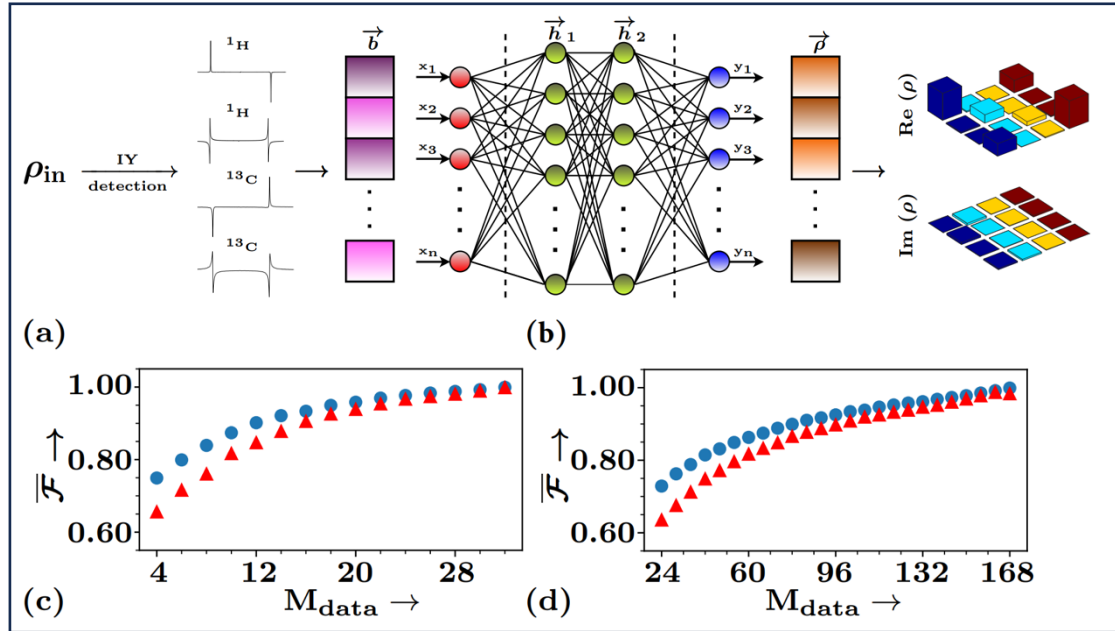
We have an unknown quantum channel $\mathcal{E}$ affect on ρ_{in} st $\rho_{out} = \mathcal{E}(\rho_{in})$

$\mathcal{E}(\rho) = \sum_{i,j} \mathcal{X}_{i,j} K_i \rho K_j^\dagger$ where $\{K\}$ is the basis operators, $\mathcal{X}$ is the learnable operator

$\Rightarrow$ We want to find $\mathcal{X}$ through 2^n measurements. Instead of that,



(1) We achieve it by an optimization process.



(2) By a neural net

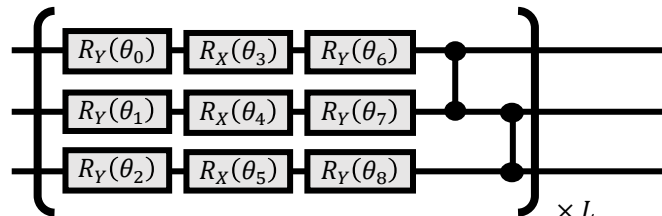
[1.5.1] Linh, H. L. D., Hai, V. T., & Ho, L. B. (2026). Advancing Quantum Process Tomography Through Quantum Compilation. *Advanced Quantum Technologies*, 9(2), e00494.

[1.5.2] Gaikwad, A., Bihani, O., Arvind, & Dorai, K. (2024). Neural-network-assisted quantum state and process tomography using limited data sets. *Physical Review A*, 109(1), 012402.

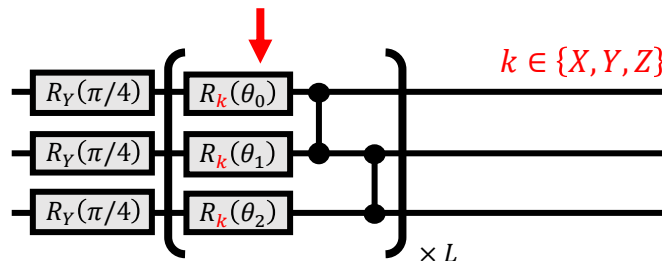
1.6. Additional informations

Barren plateaus (BP) → A method for mitigate BP

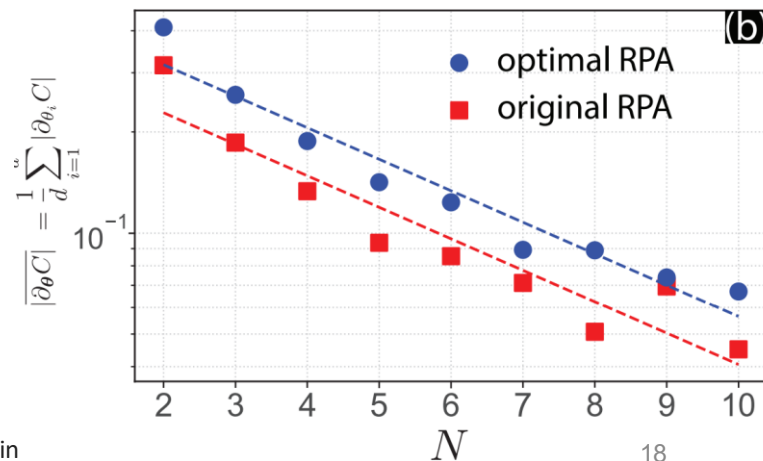
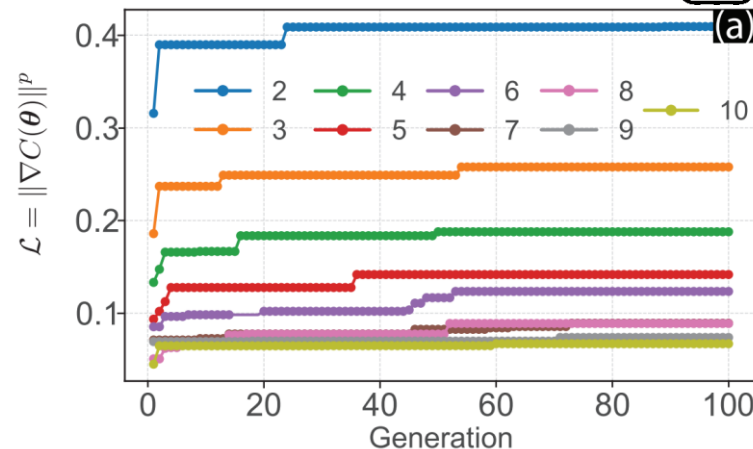
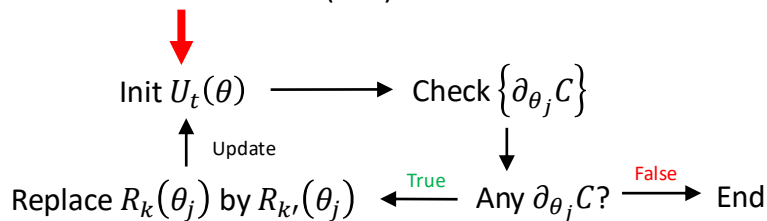
Different ansatz affect strongly to the performance



Hardware-efficient ansatz (HEA)



Random Pauli ansatz (RPA)



1.6. Additional informations

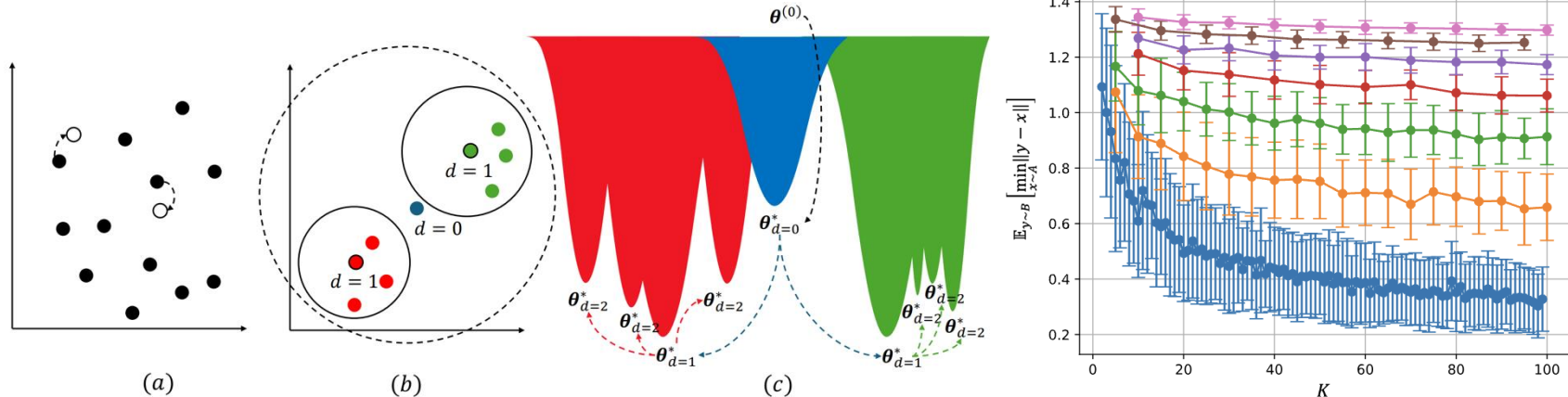
Can we apply classical techniques into quantum optimization?

Consider a set of K optimization targets denoted as $T_1, T_2, \dots, T_K$, all defined over the same search space S . Each target T_k is associated with its own cost function:

$$C(\theta_{(k)}): R^m \rightarrow [0, 1]$$

The goal of multi-target optimization is to find a set of solutions:

$$\{\theta_{(1)}^*, \theta_{(2)}^*, \dots, \theta_{(K)}^*\}$$

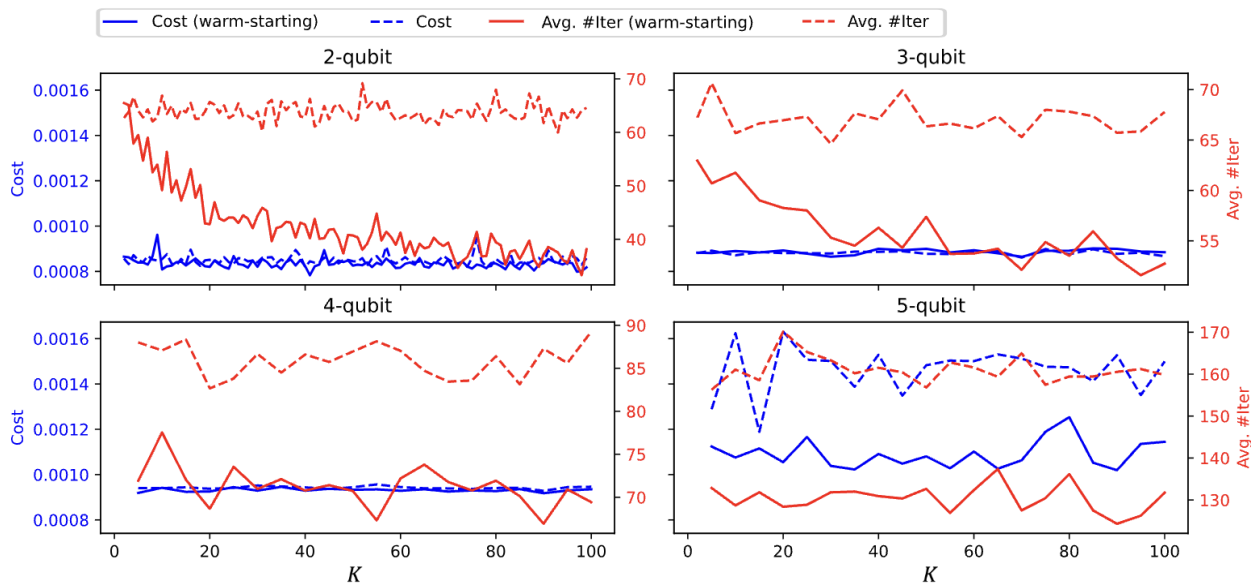


1.6. Additional informations

Can we apply classical techniques into quantum optimization?

Warm-starting method: simply set $\theta_{x(k)}^*$ as initial parameter $\theta_{y(k)}^{(0)} = \theta_{x(k)}^*$.

If $y \approx x + \epsilon$, x and y are targets, considering the first-order Taylor expansion of cost function C with respect to θ at θ_x^* and fixed y , we have:

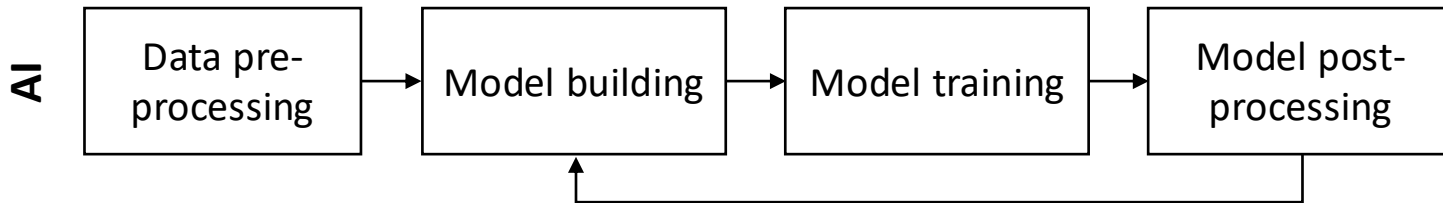


$$\tilde{\theta}_y^* \approx \theta_x^* - \frac{\nabla_{\theta} C_y(\theta_x^*)}{\|\nabla_{\theta} C_y(\theta_x^*)\|^2} C_y(\theta_x^*)$$

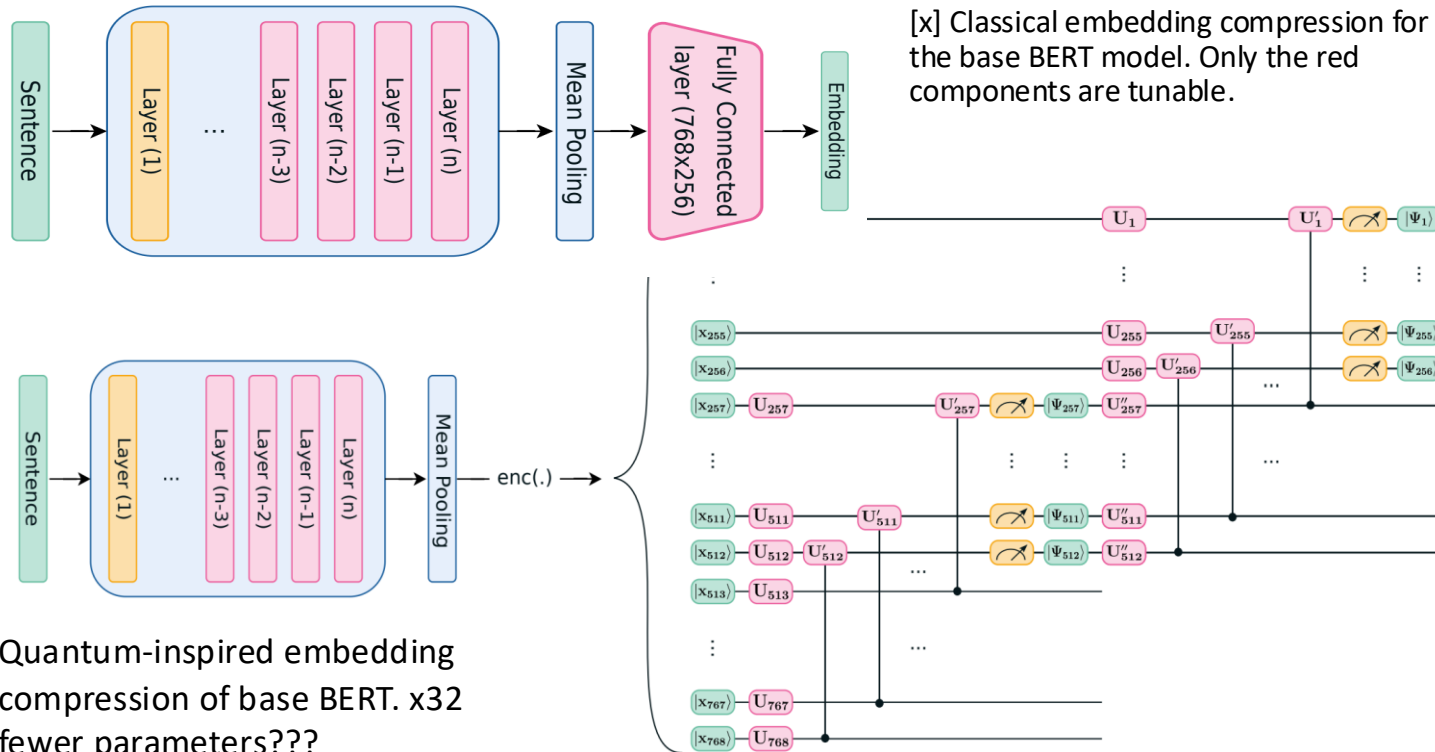


Parameter estimator

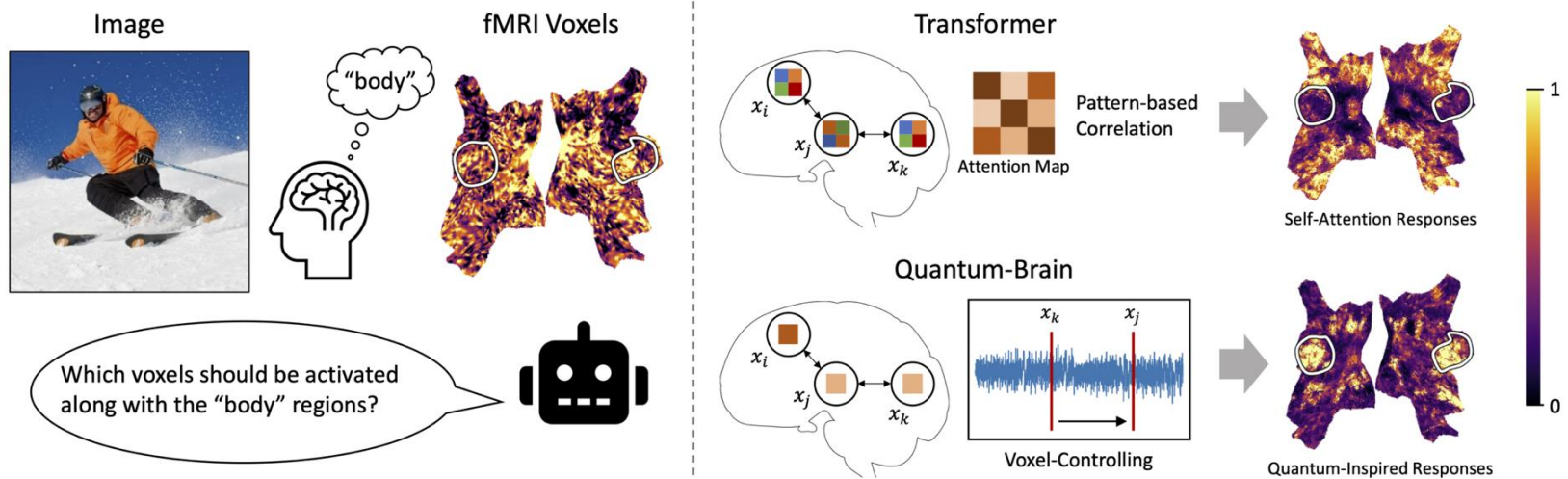
2. Quantum for AI (Quantum-inspired model)



2.1. Data pre-processing: compression task



2.2. Model building: quantum-inspired model



Quantum-Brain for Vision-Brain Understanding. Motivated by the connectivities in brain signals and entanglement in quantum theory, we propose Quantum-Brain, a quantum-inspired neural network to explore the connectivities in brain signals for vision-brain understanding. Apart from self-attention, we evaluate the entanglement between brain voxels to enhance the exploration of connectivity. **[2.2.1]**

2.2. Model building: quantum-inspired model

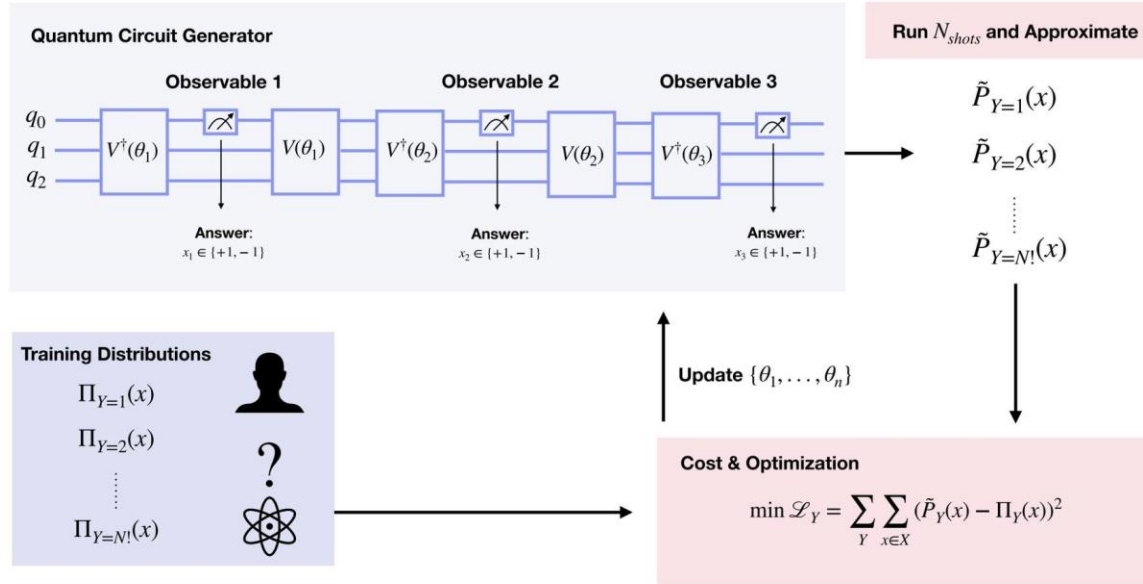
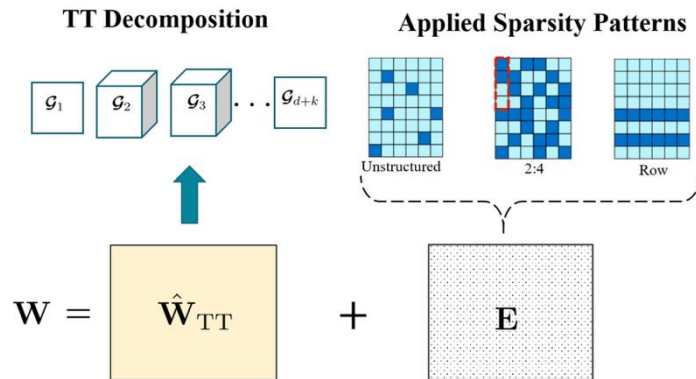


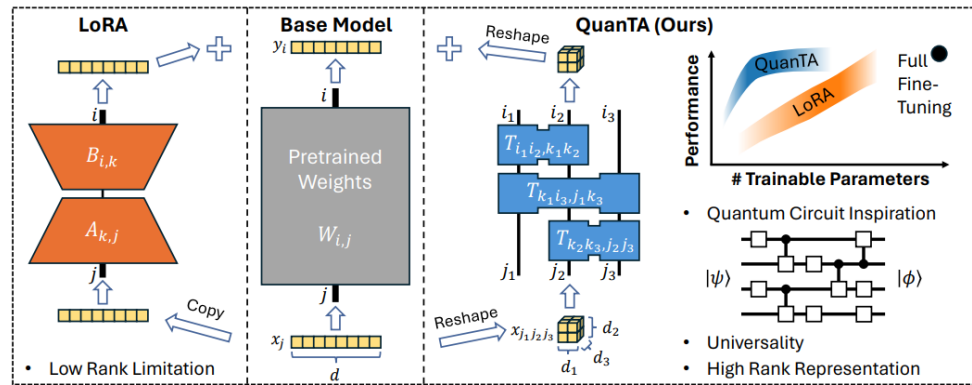
Fig. 1 An example of the multi-task quantum generative architecture for training on binary distributions with three observables. We provide a visual overview of the quantum circuit generator, containing subsequent measurement observables of the form $V(\theta_n)D_nV(\theta_n)^\dagger$. Measurements taken in the Pauli-Z basis allow one to obtain a binary eigenvalue (answer) for each trainable observable. The generator is run for a number of N_{shots} to approximate the desired number of model

output distributions $\tilde{P}_Y(x)$ over binary answer variables, with respect to the order sequence of the observables. Swapping the order sequence simply corresponds to physically swapping the parameters θ_n in the generator, which are indexed by Y . The model is trained to minimize the multi-task loss function $\mathcal{L}_Y$ that computes the LSM loss on the training distribution $\Pi_Y(x)$ and the empirical model distribution $\tilde{P}_Y(x)$ for the desired number of tasks to use for training

2.3. Post-processing: compressing by Tensor-network



(A) The sparse + low-rank tensor train representation for a weight matrix (or embedding table). [2.3.1]



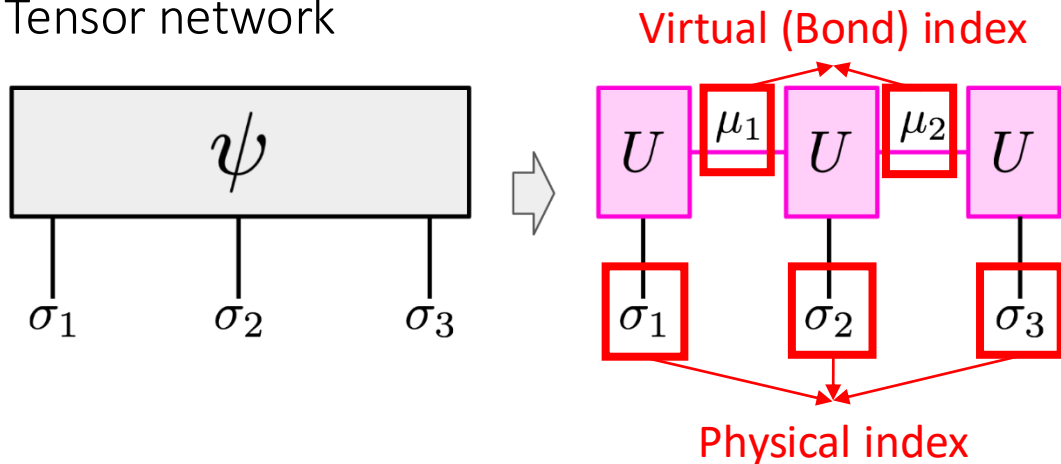
(B) Conceptual comparison of QuanTA and LoRA methods. LoRA parameterizes the weight matrix update as a outer product of two low-rank matrices, limiting its capacity. QuanTA, inspired by quantum circuits, uses tensors that operate on specific axes of the (reshaped) input, enabling high-rank parameterization.[2.3.2]

[2.3.1] Solgi, R., Zhen, K., Swaminathan, R. V., Susanj, N., Mouchtaris, T., Kunzmann, J., & Zhang, Z. (2025). *Sparse augmented tensor networks for post-training compression of large language models*. EMNLP2025

[2.3.2] Chen, Z., Dangovski, R., Loh, C., Dugan, O., Luo, D., & Soljačić, M. (2024). QuanTA: Efficient High-Rank Fine-Tuning of LLMs with Quantum-Informed Tensor Adaptation. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, & C. Zhang (Eds), *Advances in Neural Information Processing Systems* (Vol. 37, pp. 92210–92245). doi:10.52202/079017-2928

2.3. Post-processing: tensor network

Tensor network



We present 2^n statevector as $\underbrace{(2, 2, \dots, 2)}_n$ tensor

For qubit 0, we reshape $|\psi\rangle$ as row = (i_1) , col = $(i_2, \dots, i_n)$, $\psi_{i_1, (i_2, \dots, i_n)} \xrightarrow{SVD} \boxed{U} \boxed{SV^\dagger} A_1^{i_1}$

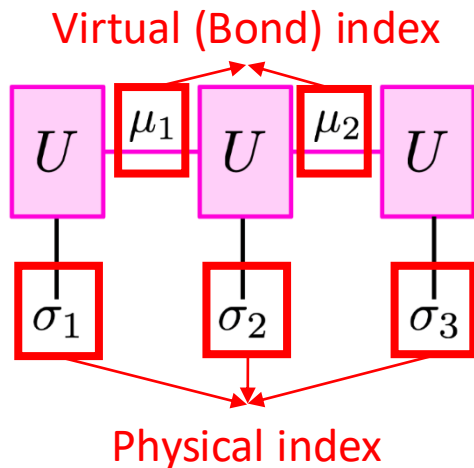
For qubit 1, we reshape $SV^\dagger$ as row = $(bond \times i_2)$, col = $(i_3, \dots, i_n)$ $\xrightarrow{SVD} A_2^{i_2} SV^\dagger$

...

Finally, we collect $\{A_1^{i_1}, A_2^{i_2}, \dots, A_n^{i_n}\}$

2.3. Post-processing: tensor network

Tensor network



In qubits system: $\sigma_j = \{0,1\}$

[2.3.3] Siddhartha Patra, Saeed S. Jahromi, Sukhbinder Singh, Roman Orus "Efficient tensor network simulation of IBM's largest quantum processors" [arXiv:2309.15642](https://arxiv.org/abs/2309.15642), 2023.

[2.3.4] H. C. Jiang, Z. Y. Weng, T. Xiang "Accurate determination of tensor network state of quantum lattice models in two dimensions" [arXiv:0806.3719](https://arxiv.org/abs/0806.3719), 2008.

$$|W_4\rangle = 1/\sqrt{4}(|0001\rangle + |0010\rangle + |0100\rangle + |1000\rangle)$$

$$|W_4\rangle \xrightarrow{(lvbond \times |\sigma_1|) \times \frac{2^n}{lvbond \times |\sigma_1|}} \psi_{i_1, (i_2 i_3 i_4)} \xrightarrow{SVD} U \Sigma V^\dagger$$

$$\Sigma = \begin{bmatrix} \sqrt{3}/2 & 0 \\ 0 & 1/2 \end{bmatrix}$$

$$\begin{matrix} U & \Sigma & V^\dagger \\ \downarrow & \downarrow & \downarrow \\ 2 \times 2 & 2 \times 2 & 2 \times 8 \end{matrix}$$

$$U(\mu_0, \sigma, \mu_1) \rightarrow \begin{cases} A_{0, \mu_0, \mu_1} = \begin{bmatrix} 1 & 1 \\ \sqrt{3} & \sqrt{6} \end{bmatrix} \\ A_{1, \mu_0, \mu_1} = \begin{bmatrix} 1 & -2 \\ \sqrt{3} & \sqrt{6} \end{bmatrix} \end{cases} \quad \begin{matrix} (l)vbond = 1 \\ (r)vbond = \{1,2\} \end{matrix}$$

For next
iteration

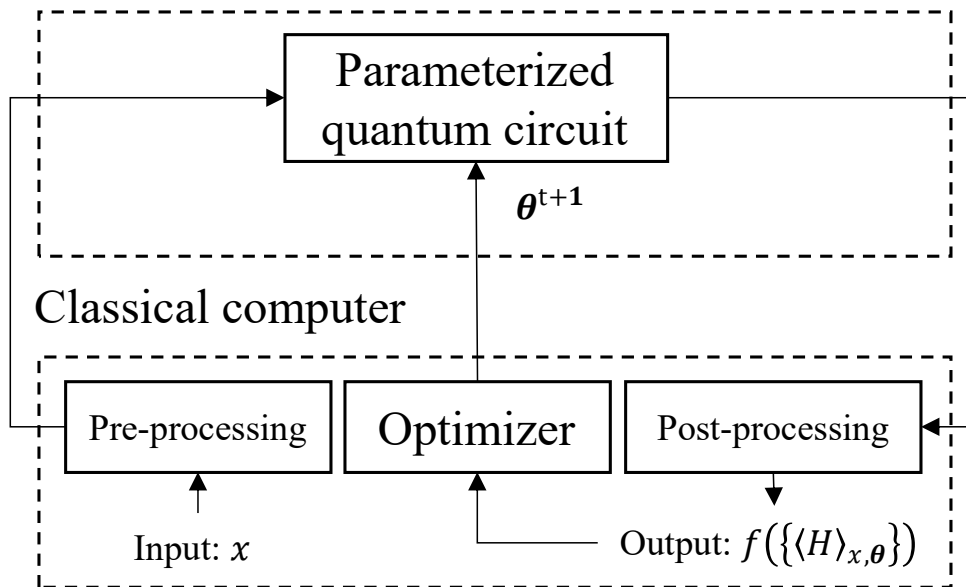
$$B = \Sigma V^\dagger$$

Collect $A_{\sigma_0, \mu_0, \mu_1}$

3. Quantum AI

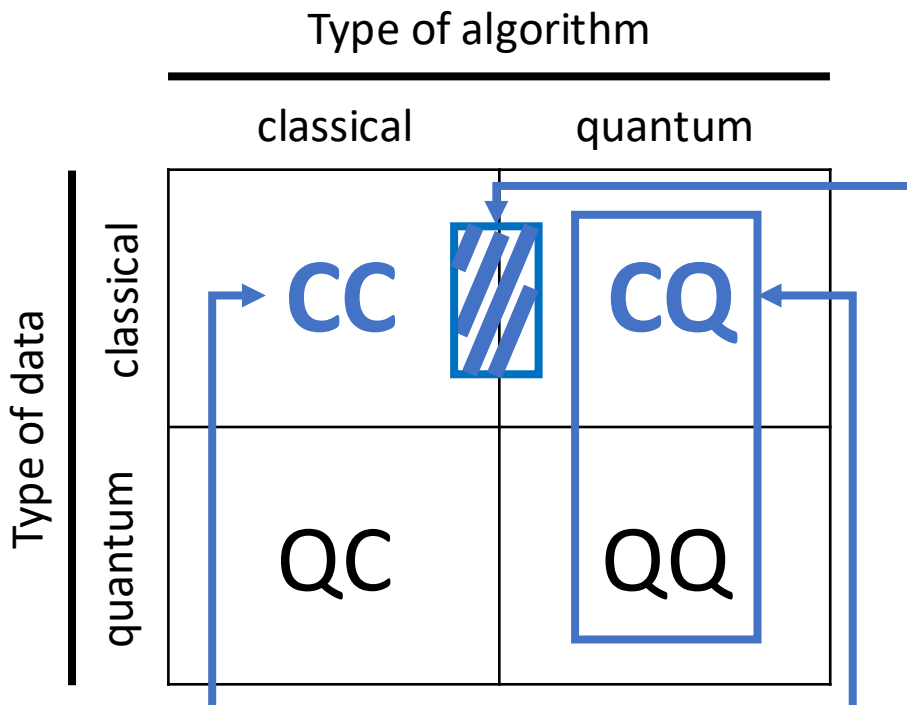
Hybrid quantum - classical model

Quantum computer



Hybrid model [3.0.1]

3. Overview



Hybrid quantum – classical learning: algorithm running on both quantum and classical computers

QML model should resolve below classical machine learning problems:

- (1) Require huge dataset
- (2) High computation resources
- (3) Lack of generality

Classical model: KNN, SVM, Deep learning, ...

Quantum Machine learning (QML): algorithm running on only quantum computer or both quantum / classical computers

3. Overview

What QML models can do?

Based on quantum advantage [3.0.3]

(1) Require huge dataset

⇒ Require less items

(2) High computation resources

⇒ Lower time complexity

(3) Lack of generality

⇒ Example: Require less #parameter than classical ML model or achieve higher accuracy on the same dataset.

**Be careful with hype
QML papers!**

[3.0.2] Simões, R. D. M., Huber, P., Meier, N., Smailov, N., Fuchsli, R. M., & Stockinger, K. (2023). Experimental evaluation of quantum machine learning algorithms. *IEEE Access*, 11, 6197-6208.

[3.0.3] Schuld, M., & Killoran, N. (2022). Is quantum advantage the right goal for quantum machine learning?. *Prx Quantum*, 3(3), 030101.

3. Overview

Type of QML model

a) Speed up version of classical ML algorithms:

- QKNN
- QKmean
- QSVM

b) Quantum analogue version of a deep learning model

- QuNN, QCNN
- QGNN
- QGAN

c) Quantum learning

→ I don't know this type

3.1. Parameterized quantum circuit (PQC)

(Main) qubits

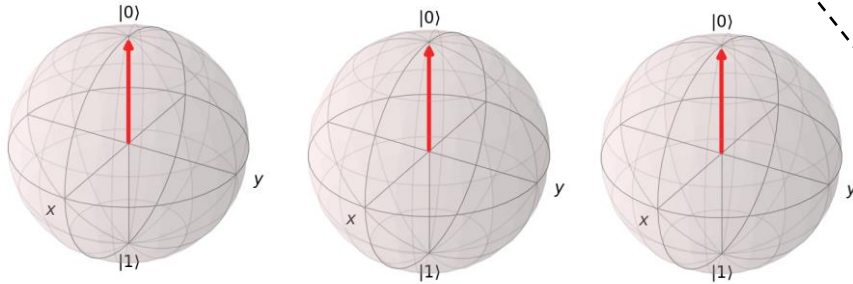
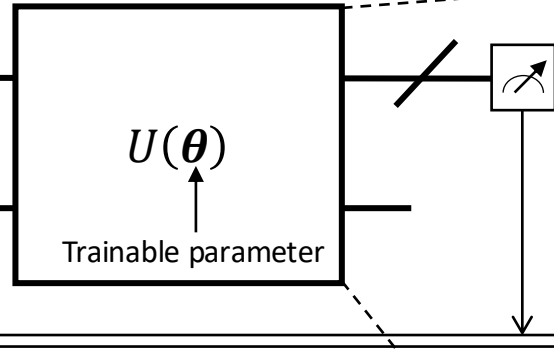
$|0\rangle^{\otimes m}$

Ancilla qubits

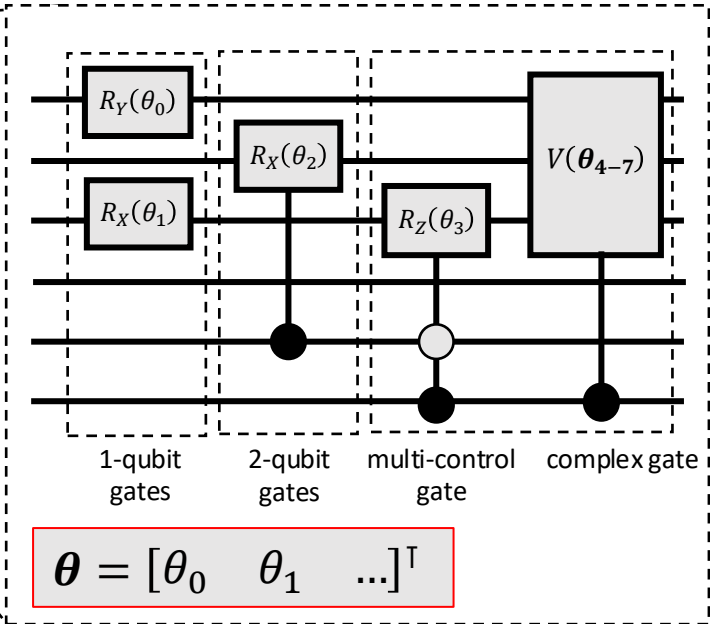
$|0\rangle^{\otimes n}$

Classical channel

c



Gate $R_i(\theta)$ rotate state θ^0 about the i -axis
where $i \in \{X, Y, Z\}$ and $\theta \in [0, 2\pi]$



[3.1.1] Benedetti, M et al, Parameterized quantum circuits as machine learning models. Quantum Science and Technology, 4(4), 043001 (2019)

3.2. Encoding

Diff encoders



Encoder \ Feature	Threshold Encoding - TE	Flexible Representation - FRQI	Novel enhanced quantum - NEQR [3.2.2]	Amplitude Encoding - AE [3.2.3] & Quantum RAM [3.2.4]	Divide – and - Conquer Encoder - DCE [3.2.5]	Universal Compilation – based variational quantum- UC – VQA [3.2.6]
#qubit	$\mathcal{O}(2^{2n})$	$\mathcal{O}(2n + 1)$	$\mathcal{O}(2n + 1)$	$\mathcal{O}(n)$	$\mathcal{O}(2^{2n})$	$\mathcal{O}(n)$
Circuit depth	$\mathcal{O}(2^{2n})$	$\mathcal{O}(2^{4n})$	$\mathcal{O}(2^{2n})$	$\mathcal{O}(2^{2n})$	$\mathcal{O}(k)$	$\mathcal{O}(\text{poly}(n)) + \mathcal{O}(\text{poly}(n))$

List of encoders

Circuit depth

Resource for θ_ϕ

[3.2.2] Zhang, Y., Lu, K., Gao, Y., & Wang, M, NEQR: a novel enhanced quantum representation of digital images. Quantum information processing, 12, 2833-2860 (2013).

[3.2.3] M. Weigold et al, Data encoding patterns for quantum computing, in Proceedings of the 27th Conference on Pattern Languages of Programs PLoP '20, (The Hillside Group, USA, 2022).

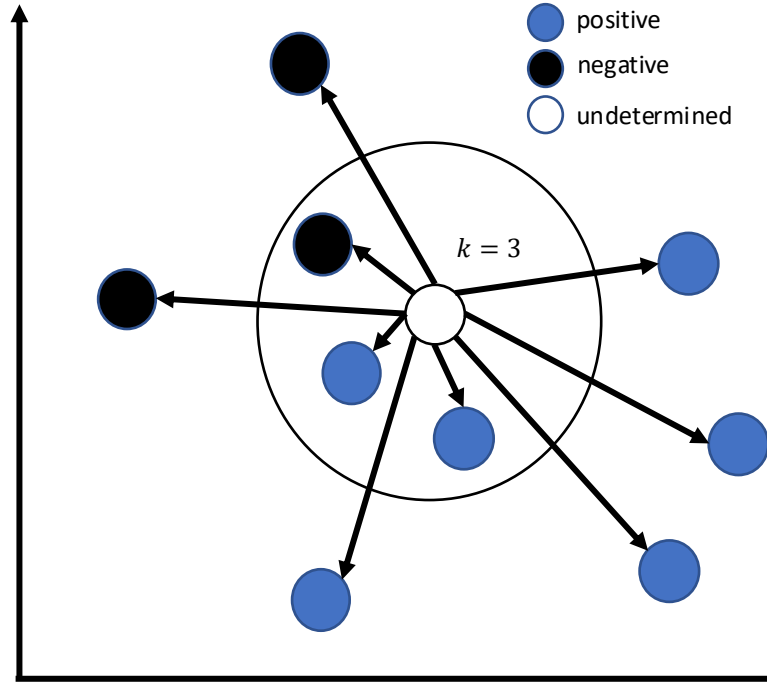
[3.2.4] V. Giovannetti et al, Quantum random access memory, Phys. Rev. Lett. 100 (Apr 2008) p. 160501.

[3.2.5] I. F. Araujo et al, A divide-and-conquer algorithm for quantum state preparation, Scientific Reports 11 (Mar 2021) p. 6329.

[3.2.6] Hai, V. T., & Ho, L. B. (2023). Universal compilation for quantum state tomography. Scientific Reports, 13(1), 3750

3.3. QML algorithms

QkNN



For each test data point a :

Step 1: Calculate $Ds_a = dist(a, x_i) \forall i = 1..n$

Step 2: Sort and get k smallest dist in Ds_a .

Step 3: Get k corresponding label and do major vote

Time complexity: $\mathcal{O}(mnn_{test})$

where:

m : dimensional of x_i

n : # train data points

n_{test} : # test data points

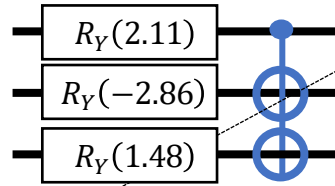
3.3. QML algorithms

QkNN



$$x = \begin{bmatrix} 1 \\ 0 \\ 2 \\ 3 \end{bmatrix} \xrightarrow{\text{Normalized}} \hat{x} = \begin{bmatrix} 1/\sqrt{14} \\ 0 \\ 2/\sqrt{14} \\ 3/\sqrt{14} \end{bmatrix}$$

Encoded



Fredkin-gate $\equiv$ SWAP

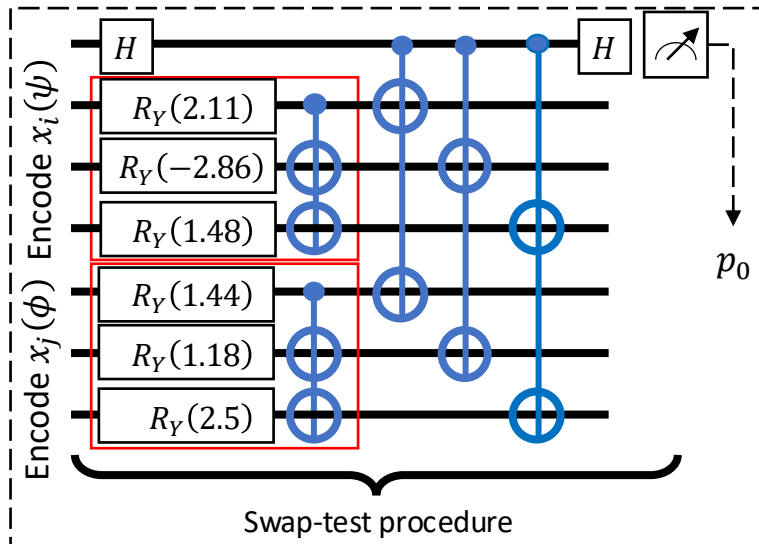
For each test data point a :

Step 1: Calculate

$$Ds_a = \text{dist}(a, x_i) \forall i = 1..n$$

Step 2: Sort and get k smallest dist in Ds_a .

Step 3: Get k corresponding label and major vote



$$\begin{aligned} |0\rangle|\psi\rangle|\phi\rangle &\xrightarrow{H_0} \frac{1}{\sqrt{2}} (|0\rangle|\psi\rangle|\phi\rangle + |1\rangle|\psi\rangle|\phi\rangle) \\ &\xrightarrow{\{SWAP_{i,j}\}} \frac{1}{\sqrt{2}} (|0\rangle|\psi\rangle|\phi\rangle + |1\rangle|\phi\rangle|\psi\rangle) \end{aligned}$$

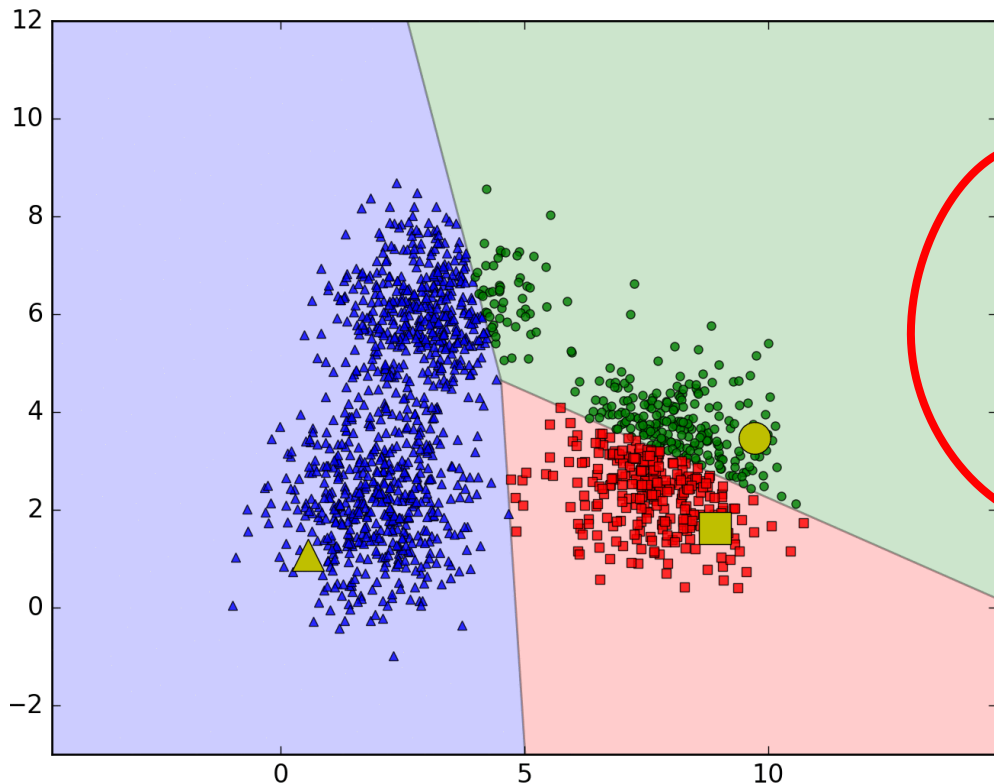
$$\xrightarrow{H_0} \frac{1}{2} (|0\rangle|\psi\rangle|\phi\rangle + |1\rangle|\phi\rangle|\psi\rangle) + \frac{1}{2} (|0\rangle|\phi\rangle|\psi\rangle - |1\rangle|\psi\rangle|\phi\rangle)$$

$$\text{So } p_0 = \frac{1}{2} (\langle\phi|\langle\psi| + \langle\psi|\langle\phi|) \frac{1}{2} (|\phi\rangle|\psi\rangle + |\psi\rangle|\phi\rangle)$$

$$\Rightarrow |\langle\psi|\phi\rangle| = 2p_0 - 1$$

3.3. QML algorithms

k -means



iteration 0: assign points to clusters

Step 1. Select k random centers.

Step 2. Assign each data point to the closest center.

If the assigned labels do not change, then stop.

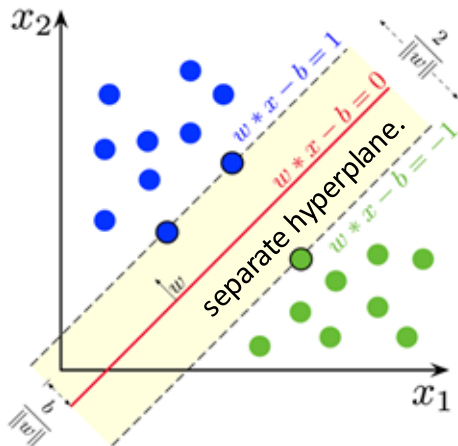
Step 3. New center = average of all assigned points

Step 4. Return to step 2.

Swap-test procedure can be applied again!

3.3. QML algorithms

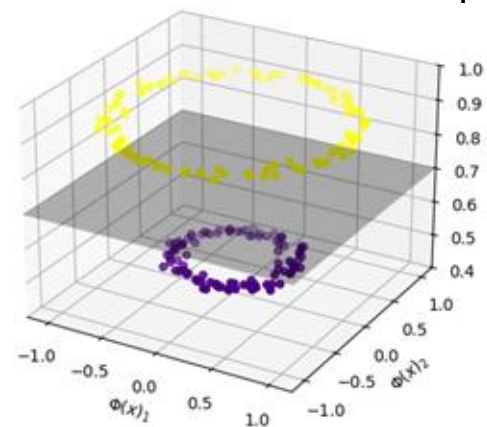
Quantum SVM



Inseparable data

$$x = [x_1 \quad x_2]^T$$

Kernel method



Separable data

$$\phi(x) = [x_1 \quad x_2 \quad x_1 x_2]^T$$

We have:

parameters

$$f^*(x) = \text{sgn}(w \cdot x \pm b),$$

with target (w^*, b^*) :

$$\min_{w,b,c} \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \epsilon_i$$

$$\max_{\beta} L(\beta) = \sum_{i=1}^n \beta_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \beta_i \beta_j y_i y_j \phi(x_i)^T \phi(x_j)$$

$$K(x_i, x_j) \equiv \phi(x_i)^T \phi(x_j)$$

Kernel function is faster!

Inner product
 $\mathcal{O}(M * ?)$ complex

3.3. QML algorithms

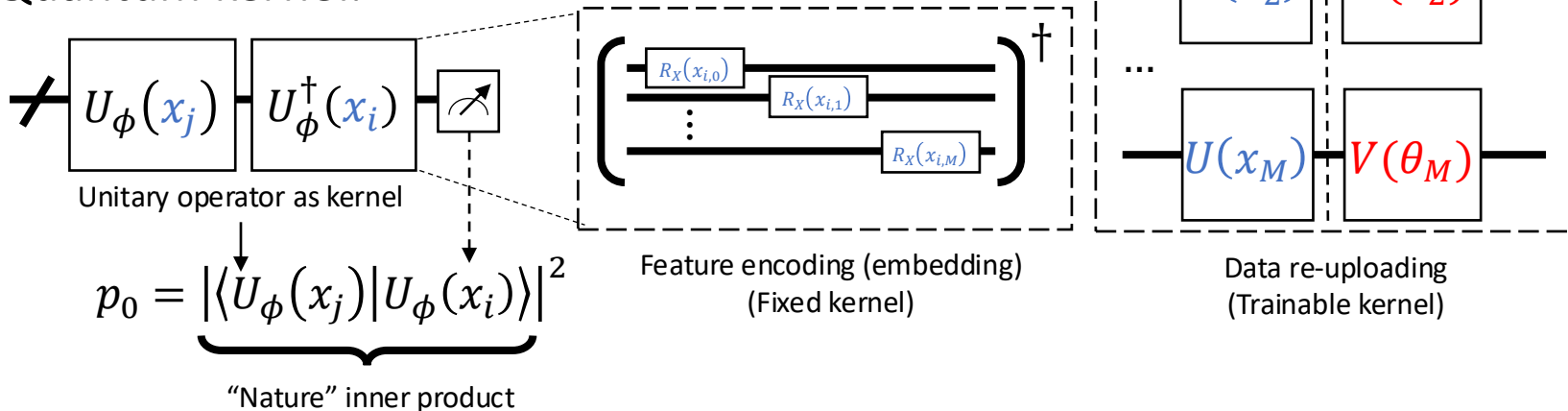
Quantum SVM

Classical kernel:

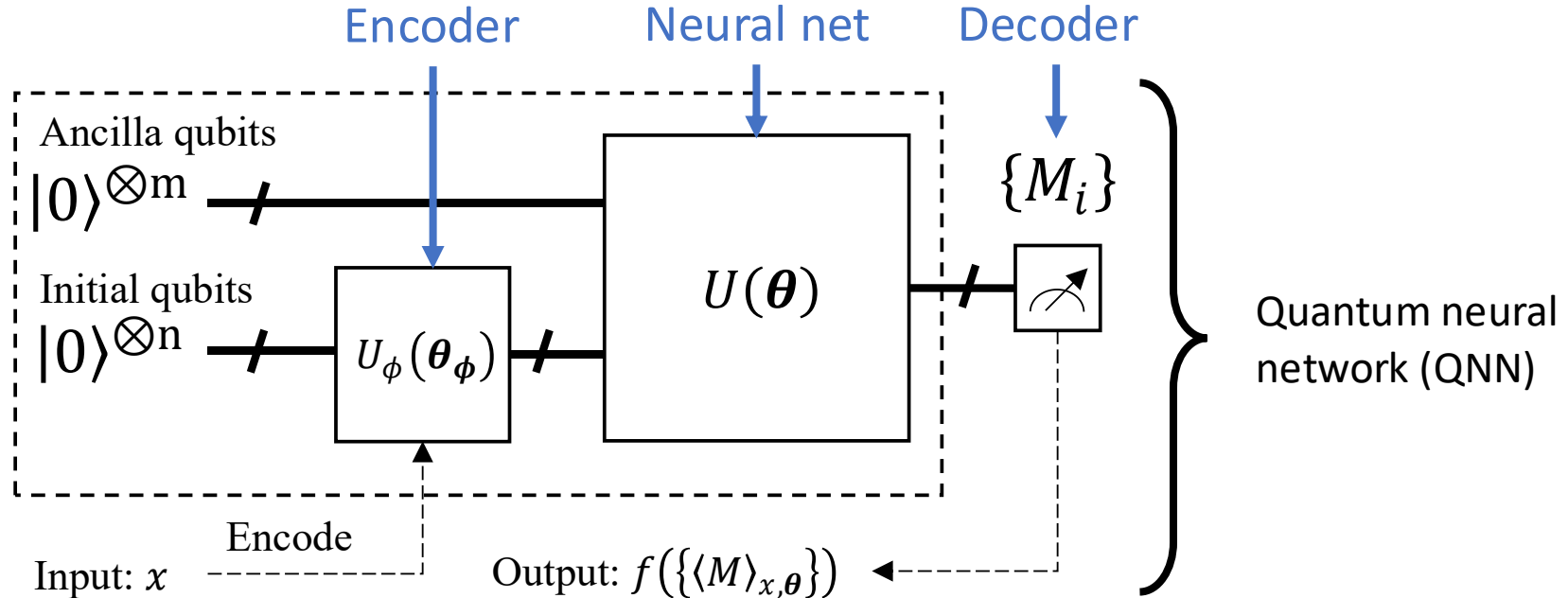
$$K_{\text{RBF}}(x_i, x_j) = \exp\left(-\gamma \|x_i - x_j\|_2^2, \gamma > 0\right)$$

$$K_{\text{Linear}}(x_i, x_j) = x_i^\top x_j$$

Quantum kernel:

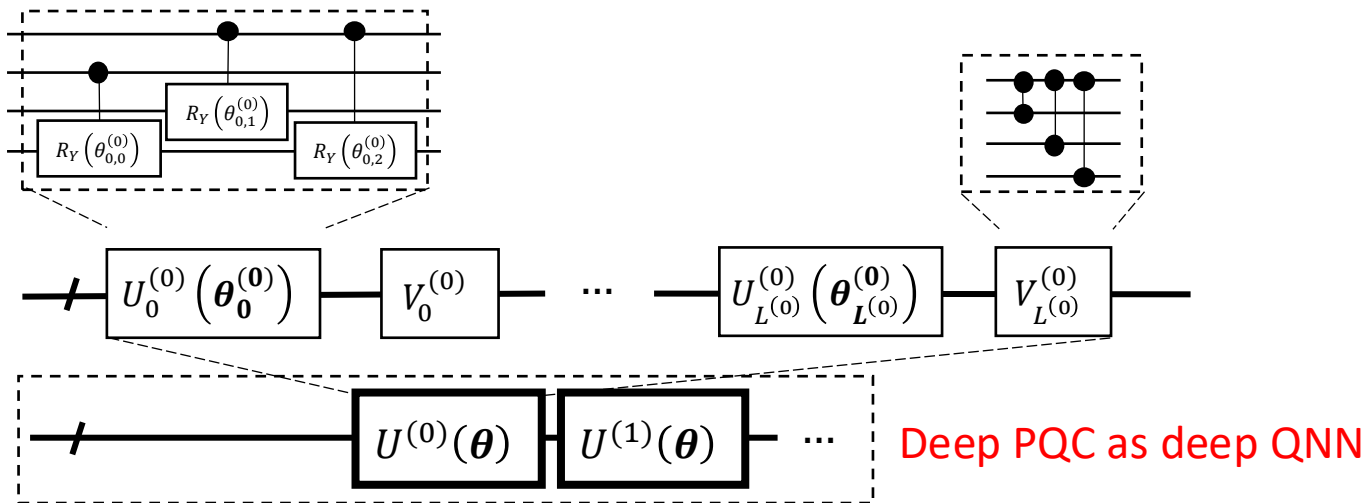


3.4. Quantum deep learning



3.4. Quantum deep learning

QNN



Note that $U(\theta, \lambda, \phi) \equiv \left(R_X(\theta) \ R_Y(\theta) \ R_Z(\theta) \right) \times K$

Then ...

Successive parameterized gates are meaningless
Entangled gates (*CX*, *MCX*, *Control-U*, ...) should be appended in the middle

3.4. Quantum deep learning

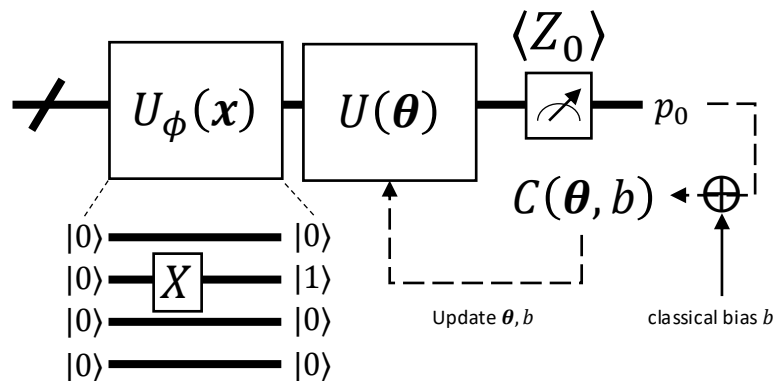
QNN



Attribute	Classical deep learning	Quantum deep learning
Main concept	Neural network	Parameterized quantum circuit (PQC)
Parameter	Weight $\{w_i\}$ in nodes	Angle value $\{\theta_i\}$ in rotation gates $\{R_X, R_Y, R_Z\}$
Gradient	Backpropagation	General parameter-shift rule and gradient tensor metric
Optimizer	Classical optimizer	Classical optimizer + gradient-free optimizer
Loss	Various type	Associated with expectation values

3.4. Quantum deep learning

Variational classifier for simple classification problem



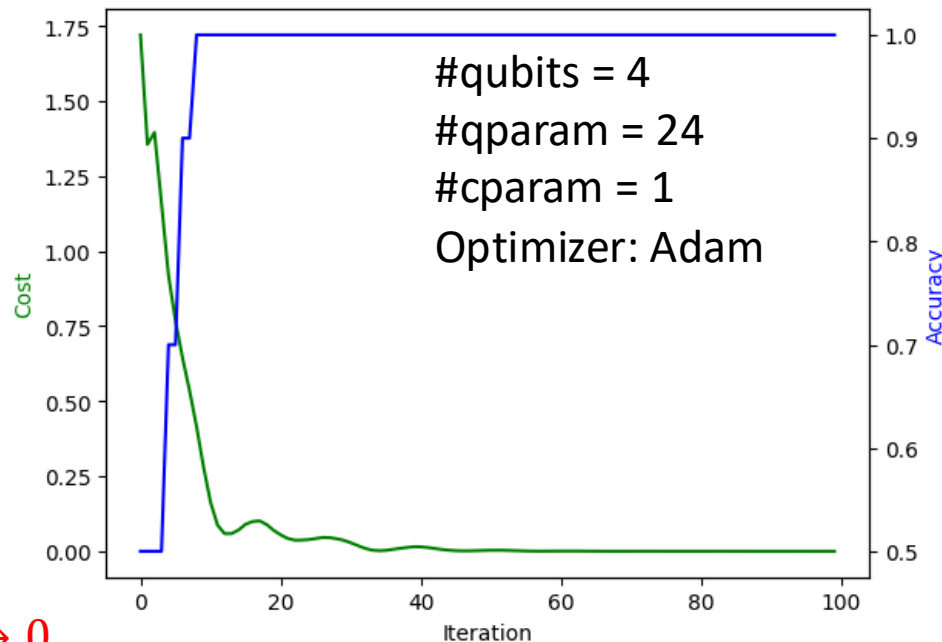
Problem:

Simulate $f: x \in \{0,1\}^{\otimes n} \rightarrow y = \bigoplus_i x_i$

Example: $f(|0100\rangle) = 1$

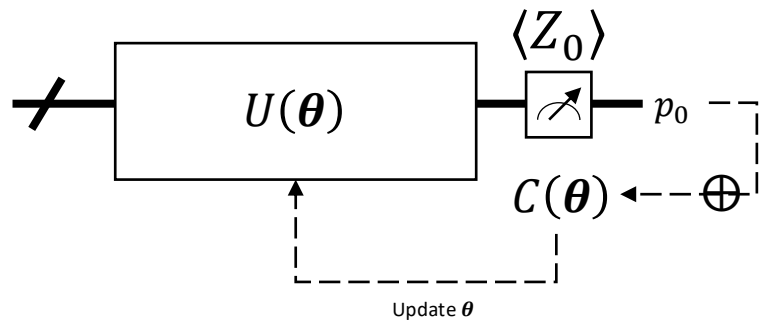
Cost function $C(\theta, b) = \frac{1}{N} \sum (\hat{y}_i - y_i)^2 \rightarrow 0$

Accuracy metric $A = \frac{1}{N} \sum \mathbb{I}(|y_i - \hat{y}_i| < \tau) \rightarrow 1$



3.4. Quantum deep learning

Variational classifier for ordinary differential equation (ODE)



Solve ODE: $y'(x) = -0.5 \sin(x) + 0.5$.

Then find $y(x)$ $\left(y(x) = \frac{1}{2}(\cos(x) + x)\right)$.

Assume that $U(\theta, x) \equiv y(x)$, then:

$$y'(x) = \frac{dU}{dx} = \frac{(U(\theta, x+\epsilon) - U(\theta, x-\epsilon))}{2\epsilon}$$

We **simulate** $y(x)$ via $U(\theta, x)$

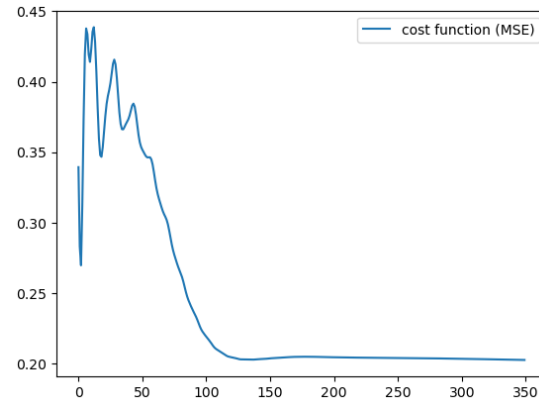
Cost function:

$$C(\theta) = [U(\theta, x_0) - y_0]^2 + \sum_j \left(\frac{dU(\theta, x_j)}{dx} - y'(x_j) \right)^2$$

Sampling from y'

$$\{\partial C(\theta) / \partial \theta_j\}$$

$$\min_{\theta} C(\theta)$$



3.4. Quantum deep learning

Variational classifier for quantum battery optimization



A quantum battery is given in the ground state of a Hamiltonian

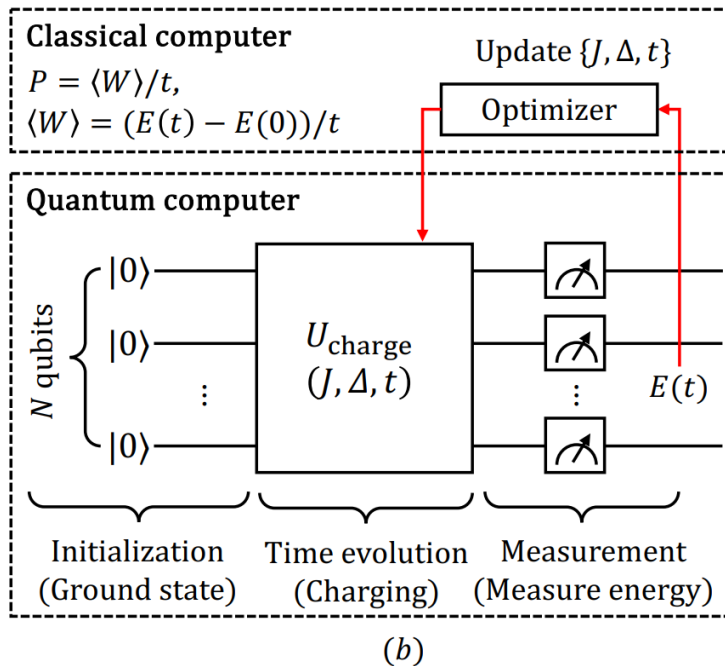
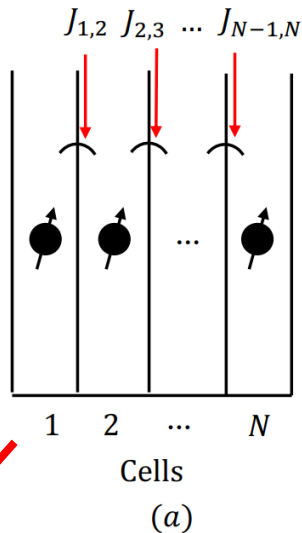
$H_0(h) = -h \sum_{j=1}^N Z_j$. The **charging Hamiltonian** is given by:

$$H_{\text{charge}}(J, \Delta, \gamma) = \sum_{j=1}^{N-1} J_j h(1 + \gamma) X_j X_{j+1} + (1 - \gamma) Y_j Y_{j+1} \quad (5.1)$$

$$- \sum_{j=1}^{N-1} \Delta_j Z_j Z_{j+1} + H_0(h)$$

Optimizing $\{J, \Delta\}$ mean
we have a better battery

How to optimize $\{J, \Delta\}$?



(a) A model of N -cell quantum battery.

(b) A quantum machine learning algorithm for optimizing battery's power.

3.4. Quantum deep learning

Variational classifier for quantum battery optimization



After **charging**, we have power (P) (performance of battery) :

$$\begin{aligned} \langle W \rangle &= \langle \psi(t) | H_0 | \psi(t) \rangle - \langle \psi(0) | H_0 | \psi(0) \rangle \\ \psi(t) &= e^{-iHt} |\psi(0)\rangle \end{aligned} \quad (5.2)$$

Calculate directly

$$P = \langle W \rangle / t$$

Can be found after survey
 $t \in [0, T]$ with $t^{k+1} \leftarrow t^k + \Delta_t$

$$P_{\max} > P \forall t?$$

We want to find “max” of $P_{\max}$

$$P_{\max}^* > P_{\max} \forall J, \Delta \longrightarrow \mathcal{L}(J, \Delta, \gamma) = -|P_{\max}|$$

$$(J|\Delta)^{t+1} = (J|\Delta)^t - \alpha \nabla \mathcal{L}$$

Update new $\{J, \Delta\}$ by
classical optimizer

$$\nabla \mathcal{L}$$

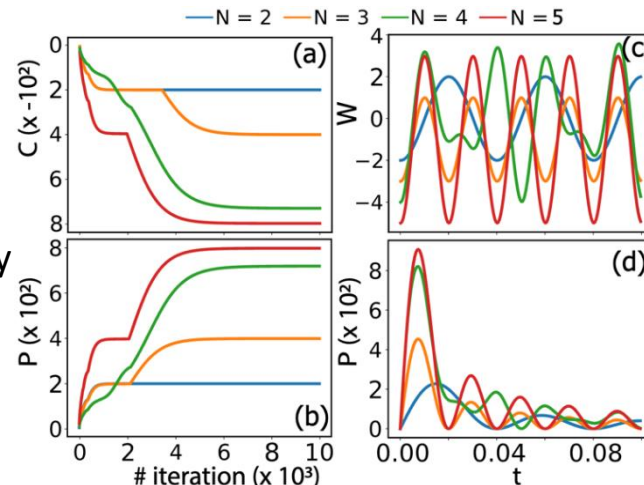
Calculate gradient

ClassicalQuantum

t : charging time

$|\psi(0)\rangle$: initial ground state

$$H_{\text{charge}} = \sum_{j=1}^{n-1} J_j (X_j X_{j+1} + Y_j Y_{j+1})$$



(a) Plot of cost value

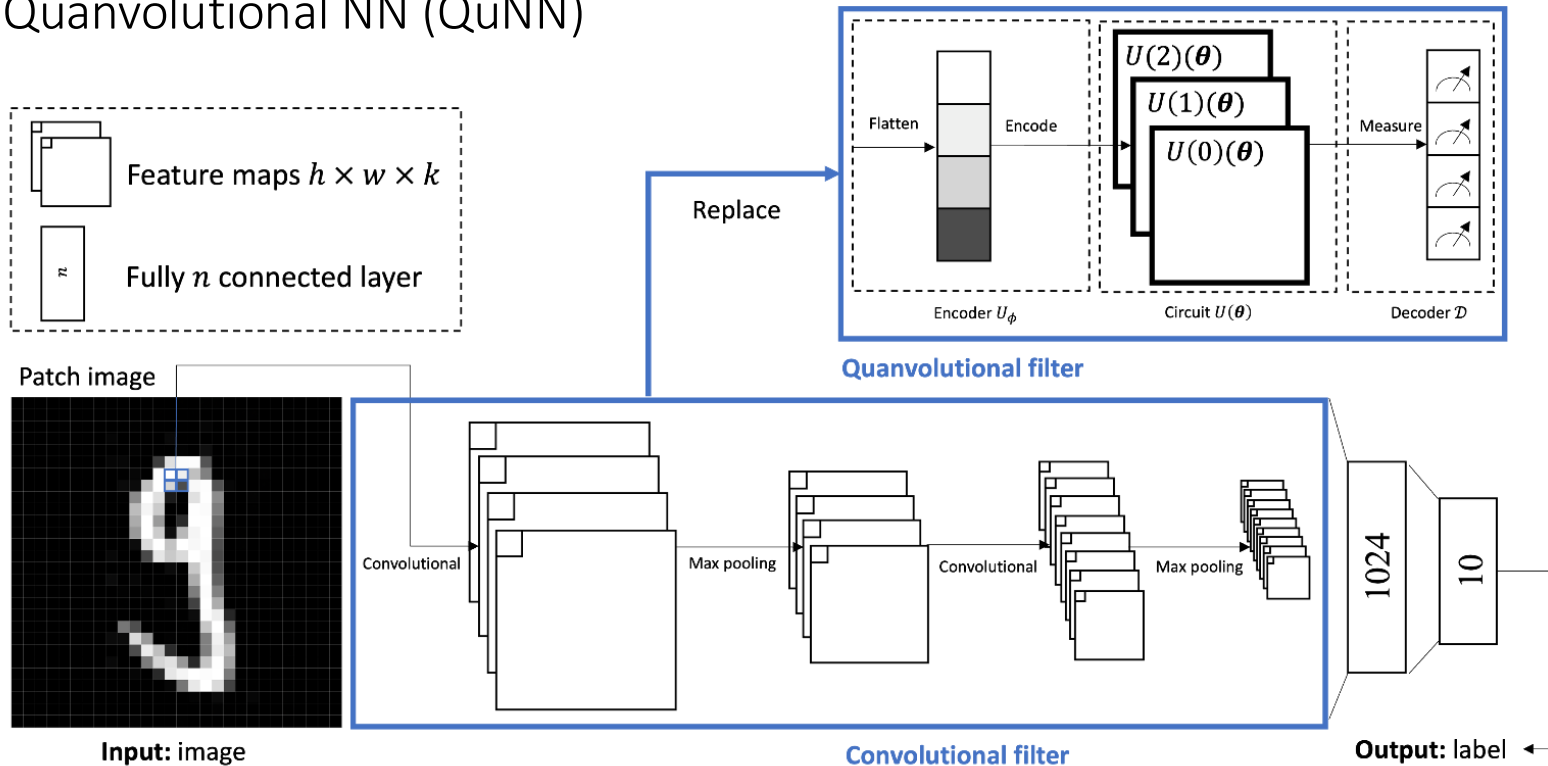
(b) Plot of power versus the iteration.

(c) Energy as a function of time.

(d) Power as a function of time.

3.4.1. Quantum deep learning

Quantum convolutional NN (QuNN)

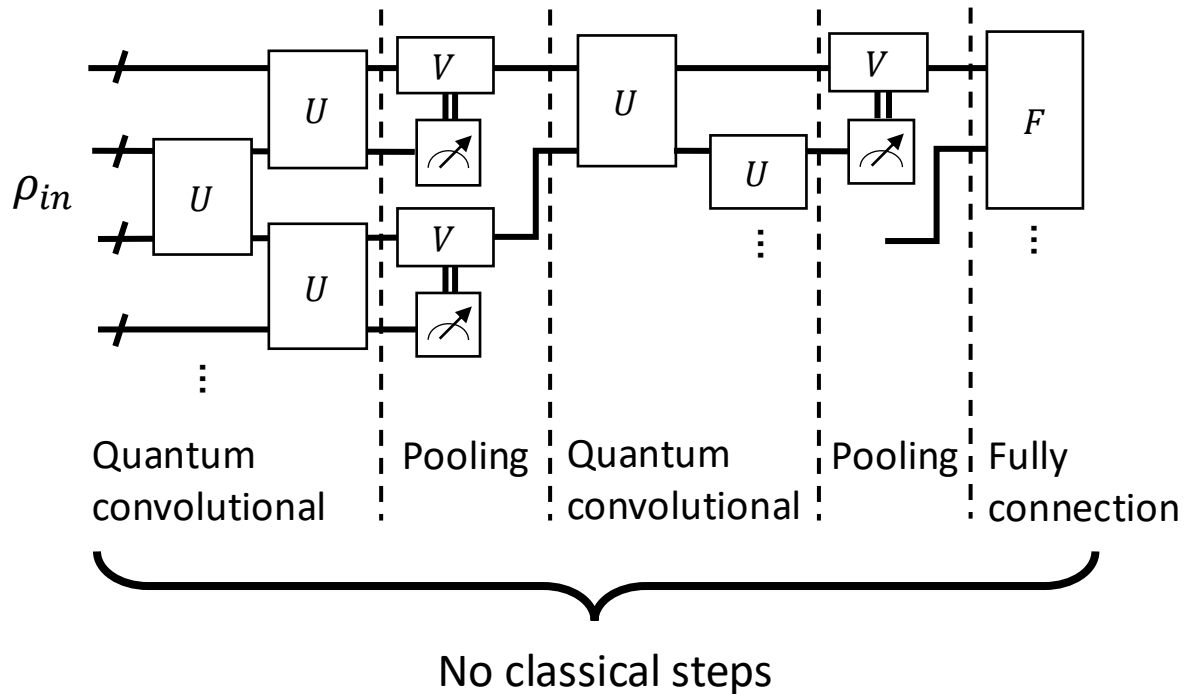


[3.4.2] Y. Lecun et al, Gradient-based learning applied to document recognition, Proceedings of the IEEE 86(11) (1998) 2278–2324

[3.4.3] T. Hur et al, Quantum convolutional neural network for classical data classification, Quantum Machine Intelligence 4 (Feb 2022) p. 3.

3.4.2. Quantum deep learning

Quantum Convolution NN (QCNN)



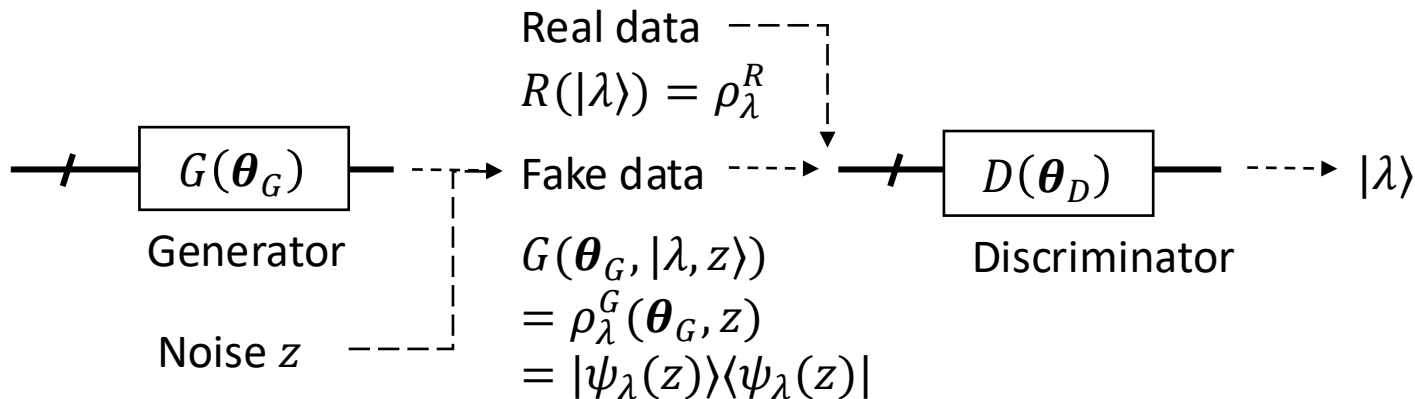
3.4.3. Quantum deep learning

Quantum Generative adversarial net (QGAN)



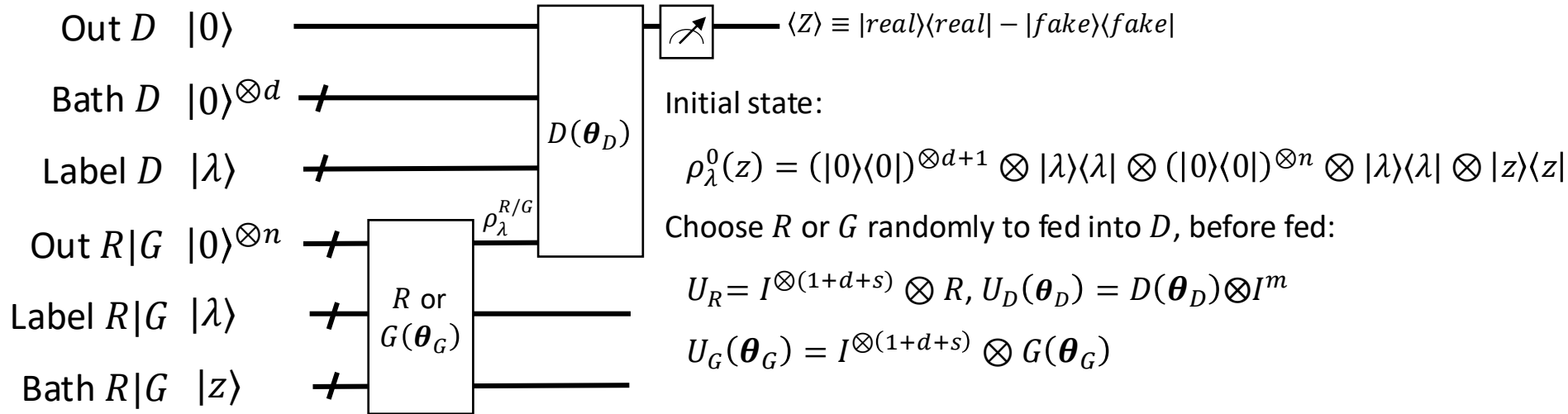
Assume real data comes from the fixed distribution $p_R(x)$ generated by process R . Adversarial task:

$$\min_{\theta_G} \max_{\theta_D} \sum_{\lambda=1} \Pr \left((D(\theta_D, |\lambda\rangle, R(|\lambda\rangle)) = |\text{real}\rangle) \right. \\ \left. \cap (D(\theta_D, |\lambda\rangle, G(\theta_G, |\lambda, z\rangle)) = |\text{fake}\rangle) \right)$$



3.4.3. Quantum deep learning

Quantum Generative adversarial net (QGAN)



After fed:

$$\rho_{\lambda}^R = U_R \rho_{\lambda}^0(z) U_R^{\dagger}$$

$$\rho_{\lambda}^G(\theta_G, z) = U_G(\theta_G) \rho_{\lambda}^0(z) U_G^{\dagger}(\theta_G)$$

After classify:

$$\rho_{\lambda}^{DR}(\theta_D) = U_D(\theta_D) \rho_{\lambda}^R U_D^{\dagger}(\theta_D)$$

$$\rho_{\lambda}^{DG}(\theta_D, \theta_G) = U_D(\theta_D) \rho_{\lambda}^G(\theta_G, z) U_D^{\dagger}(\theta_D)$$

3.4.3. Quantum deep learning

Quantum Generative adversarial net (QGAN)

Quantum cost function

$$V(\boldsymbol{\theta}_D, \boldsymbol{\theta}_G) = \frac{1}{2} + \frac{1}{2\Lambda} \sum_{\lambda} \left(\cos^2(\phi) \text{Tr} \left(Z \rho_{\lambda}^{DR}(\boldsymbol{\theta}_R) \right) - \sin^2(\phi) \text{Tr} \left(Z \rho_{\lambda}^{DG}(\boldsymbol{\theta}_R, \boldsymbol{\theta}_D) \right) \right)$$

Here the angle ϕ parametrizes the probability that R or G is used as a source

Assume that $\phi = \frac{\pi}{4}$, mean $p(R \text{ as source}) = p(G \text{ as source})$

$$V(\boldsymbol{\theta}_D, \boldsymbol{\theta}_G) = \frac{1}{2} + \frac{1}{4\Lambda} \sum_{\lambda} \text{Tr} \left(\rho_{\lambda}^{DR}(\boldsymbol{\theta}_D) - \rho_{\lambda}^{DG}(\boldsymbol{\theta}_D, \boldsymbol{\theta}_G) \right) Z$$



3.4.3. Quantum deep learning

Quantum Generative adversarial net (QGAN)

In case G perfectly generates data source, $G(\boldsymbol{\theta}_G^*) = R$

$$\Rightarrow \Pr(\text{Success } D(\boldsymbol{\theta}_D)|\boldsymbol{\theta}_D) = \frac{1}{2} \Rightarrow \boxed{\nabla_{\boldsymbol{\theta}_D} V = 0, \nabla_{\boldsymbol{\theta}_G} V = 0}$$

Stop training

D is bounded by the purity function $C(\boldsymbol{\theta}_G) = \text{tr}(\rho_R \rho_G(\boldsymbol{\theta}_G))$:

$$\frac{1}{2} C(\boldsymbol{\theta}_G) \leq \Pr(\text{Success } D(\boldsymbol{\theta}_D)|\boldsymbol{\theta}_G) \leq 1 - \frac{1}{2} C(\boldsymbol{\theta}_G)$$

$\Rightarrow$ Increase gradient value when $\Pr \approx 1/2$

3.5. Additional information

Power of QML

We will focus on PQC which is the main computation in QML

Encoder: use a very small qubit ($\log_2 N$) to loading / encoding N – dimensional normalized x .

Operator: like classical NN, the structure of $U(\theta)$ decides the model extract feature / process information good or bad.

Decoder: quantum information $\rightarrow$ classical information.

3.5. Additional information

Challenge in QML field

- Standard architecture?
- Trainability, Expressibility
- Lack of experiment resources (not enough simulated #qubit to transform toy dataset → real dataset)
- **Hype in QML**: some applied QML models are actually worse than classical models.
- Other problems

[3.5.1] Joseph Bowles, Shahnawaz Ahmed, & Maria Schuld. (2024). Better than classical? The subtle art of benchmarking quantum machine learning models.

[3.5.2] Schuld, M., & Killoran, N. (2022). Is Quantum Advantage the Right Goal for Quantum Machine Learning?. PRX Quantum, 3, 030101.

[3.5.3] Jens Eisert, & John Preskill. (2025). Mind the gaps: The fraught road to quantum advantage.

Thank you for everything!